TUGAS BESAR 1 IF3070 DASAR INTELIGENSI ARTIFISIAL

Optimasi Penempatan Kendaraan pada Geladak Kapal Kargo dengan Algoritma Local Search



Muhammad Syauqi Azizi
18224046

Rafi Azmi Zafarani
18224082

Athar Ramadhana Dermawan
18224104

DAFTAR ISI

DAFTAR ISI **2**
DAFTAR TABEL **4**
PENDAHULUAN **6**
1.1 Deskripsi Permasalahan 6
1.2 Tujuan dan Batasan Permasalahan 6
PEMBAHASAN **7**
2.1 Representasi Struktur Data 7
2.1.1 Data Kapal dan Kendaraan 7
2.1.2 Representasi State dan Penempatan Kendaraan 7
2.1.3 Format Input dan Output 8
2.2 Fungsi Objektif, Constraint, dan Neighbor 8
2.2.1 Fungsi Objektif Utama 8
2.2.2 Pemeriksaan Constraint 9
2.2.3 Inisialisasi State dan Pembentukan Neighbor 10
2.2.4 Fungsi Objektif Alternatif 10
2.3 Implementasi Algoritma Local Search 11
2.3.1 First Improvement Hill Climbing 11
2.3.2 Steepest Ascent Hill Climbing 12
2.3.3 Stochastic Hill Climbing 13
2.3.4 Hill Climbing with Sideways Move 14
2.3.5 Random Restart Hill Climbing 15
2.3.6 Simulated Annealing 16
2.3.7 Genetic Algorithm 18
2.3.8 Penanganan Multi-Kapal 20
2.3.9 Program Utama, Runner Eksperimen, dan Visualisasi 22
2.4 Hasil Eksperimen 23
2.4.1 Hasil First Improvement Hill Climbing 23
2.4.2 Hasil Steepest Ascent Hill Climbing 26
2.4.3 Hasil Stochastic Hill Climbing 29
2.4.4 Hasil Hill Climbing with Sideways Move 32
2.4.5 Hasil Random Restart Hill Climbing 35
2.4.6 Hasil Simulated Annealing 39
2.4.7 Hasil Genetic Algorithm dengan Variasi Generasi 42
2.4.8 Hasil Genetic Algorithm dengan Variasi Populasi 51
2.4.9 Hasil Fungsi Objektif Alternatif 60
2.4.10 Hasil Penempatan Multi-Kapal 60
2.5 Analisis Hasil Eksperimen 61
2.5.1 Kualitas Solusi dan Kedekatan dengan Optimum 61
2.5.2 Perbandingan Lima Varian Hill Climbing 62
2.5.3 Perbandingan Hill Climbing, Simulated Annealing, dan Genetic Algorithm 62
2.5.4 Perbandingan Durasi dan Konsistensi Hasil 62
2.5.5 Pengaruh Generasi dan Populasi pada Genetic Algorithm 63

2.5.6 Analisis Fungsi Objektif Alternatif dan Multi-Kapal 63
KESIMPULAN DAN SARAN 64
3.1 Kesimpulan 64
3.2 Saran 64
PEMBAGIAN TUGAS 64
REFERENSI 65
LAMPIRAN A FORM PENGGUNAAN GENERATIVE AI 66
LAMPIRAN B HASIL ATAU LISTING TAMBAHAN 67

DAFTAR TABEL

Tabel 1. Representasi Entitas Kapal dan Kendaraan (hlm. 7)
Tabel 2. Struktur State dan Placement (hlm. 7)
Tabel 3. Format Input dan Output Program (hlm. 8)
Tabel 4. Implementasi Fungsi Objektif Utama (hlm. 8)
Tabel 5. Implementasi Pemeriksaan Constraint (hlm. 9)
Tabel 6. Implementasi Inisialisasi dan Neighbor (hlm. 10)
Tabel 7. Implementasi Fungsi Objektif Alternatif (hlm. 10)
Tabel 8. Implementasi First Improvement Hill Climbing (hlm. 11)
Tabel 9. Implementasi Steepest Ascent Hill Climbing (hlm. 12)
Tabel 10. Implementasi Stochastic Hill Climbing (hlm. 13)
Tabel 11. Implementasi Hill Climbing with Sideways Move (hlm. 14)
Tabel 12. Implementasi Random Restart Hill Climbing (hlm. 15)
Tabel 13. Implementasi Simulated Annealing (hlm. 16)
Tabel 14. Implementasi Genetic Algorithm (hlm. 18)
Tabel 15. Implementasi Penanganan Multi-Kapal (hlm. 20)
Tabel 16. Implementasi Program Utama, Runner Eksperimen, dan Visualisasi (hlm. 22)
Tabel 17. Hasil First Improvement Hill Climbing , Percobaan 1 (hlm. 23)
Tabel 18. Hasil First Improvement Hill Climbing , Percobaan 2 (hlm. 25)
Tabel 19. Hasil First Improvement Hill Climbing , Percobaan 3 (hlm. 26)
Tabel 20. Hasil Steepest Ascent Hill Climbing , Percobaan 1 (hlm. 26)
Tabel 21. Hasil Steepest Ascent Hill Climbing , Percobaan 2 (hlm. 28)
Tabel 22. Hasil Steepest Ascent Hill Climbing , Percobaan 3 (hlm. 29)
Tabel 23. Hasil Stochastic Hill Climbing , Percobaan 1 (hlm. 29)
Tabel 24. Hasil Stochastic Hill Climbing , Percobaan 2 (hlm. 31)
Tabel 25. Hasil Stochastic Hill Climbing , Percobaan 3 (hlm. 32)
Tabel 26. Hasil Hill Climbing with Sideways Move , Percobaan 1 (hlm. 32)
Tabel 27. Hasil Hill Climbing with Sideways Move , Percobaan 2 (hlm. 34)
Tabel 28. Hasil Hill Climbing with Sideways Move , Percobaan 3 (hlm. 35)
Tabel 29. Hasil Random Restart Hill Climbing , Percobaan 1 (hlm. 35)
Tabel 30. Hasil Random Restart Hill Climbing , Percobaan 2 (hlm. 37)
Tabel 31. Hasil Random Restart Hill Climbing , Percobaan 3 (hlm. 38)
Tabel 32. Hasil Simulated Annealing , Percobaan 1 (hlm. 39)
Tabel 33. Hasil Simulated Annealing , Percobaan 2 (hlm. 40)
Tabel 34. Hasil Simulated Annealing , Percobaan 3 (hlm. 41)
Tabel 35. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 1, Percobaan 1 (50 generasi; 30 populasi) (hlm. 42)
Tabel 36. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 1, Percobaan 2 (50 generasi; 30 populasi) (hlm. 43)
Tabel 37. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 1, Percobaan 3 (50 generasi; 30 populasi) (hlm. 44)
Tabel 38. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 2, Percobaan 1 (100 generasi; 30 populasi) (hlm. 45)
Tabel 39. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 2, Percobaan 2 (100 generasi; 30 populasi) (hlm. 46)
Tabel 40. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 2, Percobaan 3 (100 generasi; 30 populasi) (hlm. 47)
Tabel 41. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 3, Percobaan 1 (200 generasi; 30 populasi) (hlm. 48)
Tabel 42. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 3, Percobaan 2 (200 generasi; 30 populasi) (hlm. 49)
Tabel 43. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 3, Percobaan 3 (200 generasi; 30 populasi) (hlm. 50)

Tabel 44. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 1, Percobaan 1 (10 populasi; 100 generasi) (hlm. 51)
Tabel 45. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 1, Percobaan 2 (10 populasi; 100 generasi) (hlm. 52)
Tabel 46. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 1, Percobaan 3 (10 populasi; 100 generasi) (hlm. 53)
Tabel 47. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 2, Percobaan 1 (30 populasi; 100 generasi) (hlm. 54)
Tabel 48. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 2, Percobaan 2 (30 populasi; 100 generasi) (hlm. 55)
Tabel 49. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 2, Percobaan 3 (30 populasi; 100 generasi) (hlm. 56)
Tabel 50. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 3, Percobaan 1 (50 populasi; 100 generasi) (hlm. 57)
Tabel 51. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 3, Percobaan 2 (50 populasi; 100 generasi) (hlm. 58)
Tabel 52. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 3, Percobaan 3 (50 populasi; 100 generasi) (hlm. 59)
Tabel 53. Hasil Fungsi Objektif Alternatif (hlm. 60)
Tabel 54. Hasil Penempatan Multi-Kapal (hlm. 60)
Tabel 55. Analisis Kualitas Solusi dan Kedekatan dengan Optimum (hlm. 61)
Tabel 56. Analisis Perbandingan Lima Varian Hill Climbing (hlm. 62)
Tabel 57. Analisis Perbandingan Hill Climbing, Simulated Annealing, dan Genetic Algorithm (hlm. 62)
Tabel 58. Analisis Perbandingan Durasi dan Konsistensi Hasil (hlm. 62)
Tabel 59. Analisis Pengaruh Generasi dan Populasi pada Genetic Algorithm (hlm. 63)
Tabel 60. Analisis Fungsi Objektif Alternatif dan Multi-Kapal (hlm. 63)
Tabel 61. Pembagian Tugas Anggota Kelompok (hlm. 64)

BAB I

PENDAHULUAN

1.1 Deskripsi Permasalahan

Pada tugas ini, kami mengembangkan program untuk mengoptimalkan penempatan kendaraan pada geladak kapal kargo dengan tujuan memaksimalkan total biaya pengiriman (*shipping fee*) dari kendaraan yang berhasil dimuat. Setiap kendaraan memiliki atribut berupa dimensi, berat, biaya pengiriman, dan estimasi waktu kedatangan (*Estimated Time of Arrival* atau ETA). Penempatan kendaraan direpresentasikan menggunakan koordinat pada grid serta orientasinya. Dalam menentukan posisi kendaraan, kami menerapkan sejumlah batasan, yaitu kendaraan harus berada sepenuhnya di dalam area geladak, tidak boleh saling bertumpang tindih, dan total berat muatan tidak boleh melebihi kapasitas kapal. Kendaraan yang tidak dapat dimuat tetap disimpan dalam *state* dengan status outside. Program yang kami kembangkan juga mendukung penempatan kendaraan pada satu kapal maupun beberapa kapal sekaligus.

1.2 Tujuan dan Batasan Permasalahan

Pada tugas ini, kami melakukan pengujian untuk mengevaluasi penerapan model *state*, *constraint*, dan fungsi objektif, serta membandingkan kinerja lima varian Hill Climbing, Simulated Annealing, dan Genetic Algorithm. Pengujian dilakukan menggunakan dataset tambahan berupa satu geladak kapal berukuran 6 × 7 dengan kapasitas berat 20 dan delapan kendaraan (A–H). Untuk melihat konsistensi hasil dari setiap algoritma, kami melakukan tiga kali percobaan menggunakan *seed* 2027, 2028, dan 2029. Khusus Genetic Algorithm, kami juga menguji pengaruh jumlah generasi dan ukuran populasi terhadap hasil optimasi. Variasi yang digunakan meliputi 50, 100, dan 200 generasi dengan populasi tetap sebanyak 30 individu, serta populasi 10, 30, dan 50 individu dengan jumlah generasi tetap sebanyak 100. Setiap konfigurasi diuji sebanyak tiga kali. Selain pengujian utama, kami melakukan eksperimen tambahan menggunakan fungsi objektif alternatif pada dataset yang sama serta menguji penempatan kendaraan pada armada dua kapal menggunakan contoh input dari *repository*.

BAB II

PEMBAHASAN

2.1 Representasi Struktur Data

2.1.1 Data Kapal dan Kendaraan

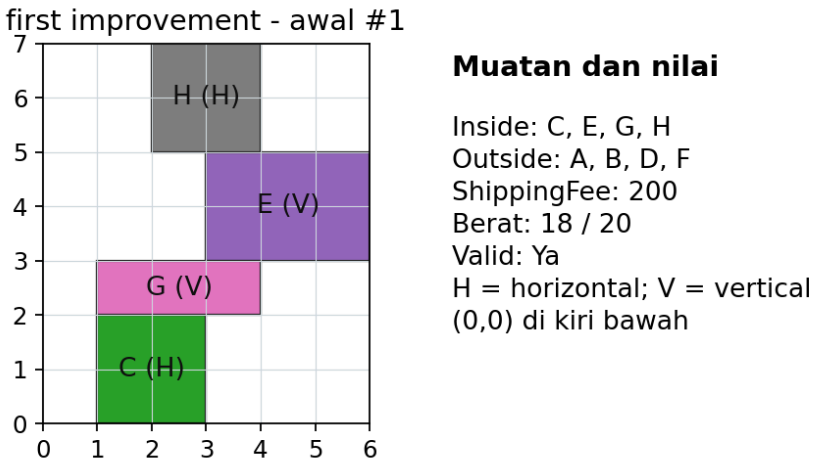
Tabel 1. Representasi Entitas Kapal dan Kendaraan

Kategori	Nama atau Atribut	Keterangan
Ship	width, length	Ukuran geladak dalam sel grid; input/example.json: 10 x 20.
Ship	max_capacity	Batas total weight kendaraan di dalam kapal; contoh 500.
Vehicle	id, width, length	ID unik dan ukuran kendaraan sebelum rotasi.
Vehicle	shipping_fee, weight	Tarif yang dijumlahkan sebagai objektif utama dan beban untuk batas kapasitas.
Vehicle	eta	Sisa waktu kedatangan; dipakai pada salah satu objektif alternatif.

2.1.2 Representasi State dan Penempatan Kendaraan

Tabel 2. Struktur State dan Placement

Kategori	Nama atau Atribut	Keterangan
Orientation	horizontal / vertical	Vertikal menukar width dan length saat menghitung ukuran terpasang.
Placement	vehicle, inside, x, y	Memetakan setiap kendaraan ke dalam atau luar kapal; koordinat hanya diperlukan saat inside.
Placement	orientation, ship_id	Orientasi dan tujuan kapal; ship_id terisi pada FleetState.
State	ship, placements	Menyimpan semua kendaraan, termasuk yang tidak dimuat, serta objective_value dan total_weight.
FleetState	ships, placements	Perluasan untuk armada multi-kapal; bukan fokus eksperimen HC satu kapal di bagian ini.

Area Gambar atau Lampiran


Gambar 1. Contoh Visualisasi State

Gambar 1 memperlihatkan contoh state awal acak pada dataset eksperimen bagian A, B, C, D, E, F, G, dan H tetap tercatat walaupun sebagian berada di luar kapal. Label H atau V menunjukkan orientasi, sedangkan bagian kanan menunjukkan objective, berat, dan validitas ya atau tidaknya.

2.1.3 Format Input dan Output

Tabel 3. Format Input dan Output Program

Kategori	Nama atau Atribut	Keterangan
Input JSON	ship atau ships	Satu objek ship atau array ships, tepat salah satunya; tiap kapal memiliki dimensi dan maxCapacity.
Input JSON	vehicles	Array objek kendaraan dengan ID unik, width, length, shippingFee, weight, ETA.
Pembacaan	load_problem(path)	Memvalidasi struktur, angka finite, dimensi integer positif, dan ID kendaraan unik.
Output	State / hasil hill_climb	initial_state, final_state, best_objective, iterations, history, duration, serta metadata varian.

2.2 Fungsi Objektif, Constraint, dan Neighbor

2.2.1 Fungsi Objektif Utama

Tabel 4. Implementasi Fungsi Objektif Utama

Kategori	Nama	Deskripsi
Parameter	mode = shipping_fee	Mode baku yang memaksimalkan jumlah ShippingFee kendaraan inside.
Variabel	state.inside_placements	Hanya kendaraan yang dimuat memberi kontribusi.
Method	calculate_objective	Menjumlah score(placement) untuk semua kendaraan di dalam.

Kategori	Nama	Deskripsi
Proses	sum(shipping_fee)	f(s) = jumlah shipping_fee kendaraan inside; kendaraan outside bernilai nol.
Return	float	Nilai objektif state; with_updated_metrics turut memperbarui total_weight.

Objektif utama adalah f(s) = Σ shipping_fee(v) untuk seluruh kendaraan v yang berada di kapal. Pada multi-kapal, penjumlahan tetap dilakukan atas seluruh kendaraan inside. Nilai ini hanya dibandingkan pada state yang valid.

Cuplikan kode: src/core/objective.py, baris 27-45

```
27  def calculate_objective(state: SearchState, mode: str = "shipping_fee") -> float:
28      try:
29          score = OBJECTIVE_MODES[mode]
30      except KeyError as error:
31          choices = ", ".join(OBJECTIVE_MODES)
32          raise ValueError(f"Unknown objective mode: {mode}. Choose from {choices}") from error
33      return sum(score(placement) for placement in state.inside_placements)
34
35
36  def calculate_total_weight(state: SearchState) -> float:
37      return sum(placement.vehicle.weight for placement in state.inside_placements)
38
39
40  def with_updated_metrics(state: SearchState, mode: str = "shipping_fee") -> SearchState:
41      return replace(
42          state,
43          objective_value=calculate_objective(state, mode),
44          total_weight=calculate_total_weight(state),
45      )
```

2.2.2 Pemeriksaan Constraint

Tabel 5. Implementasi Pemeriksaan Constraint

Kategori	Nama	Deskripsi
Batas	is_inside_boundary	x,y bilangan nonnegatif dan seluruh bidang kendaraan berada di geladak.
Tabrakan	placements_overlap	Menguji perpotongan persegi panjang pada sumbu x dan y.
Kapasitas	within_capacity	Jumlah weight kendaraan inside tidak melebihi max_capacity.
Gabungan	is_valid_state	ID unik, batas geladak, tanpa overlap, dan kapasitas terpenuhi.
Penggunaan	candidate is None	move/rotate/swap menolak state tidak valid sebelum menjadi neighbor.

Cuplikan kode: src/core/constraints.py, baris 46-59

```
46  def within_capacity(state: SearchState) -> bool:
47      if isinstance(state, FleetState):
48          return state.has_valid_assignments() and all(
49              within_capacity(state.ship_state(ship_id)) for ship_id in state.ships
50          )
51      weight = sum(placement.vehicle.weight for placement in state.inside_placements)
52      return weight <= state.ship.max_capacity
53
54
55  def is_valid_state(state: SearchState) -> bool:
56      ids = [placement.vehicle.id for placement in state.placements]
57      if len(ids) != len(set(ids)):
58          return False
59      return is_inside_boundary(state) and not has_overlap(state) and within_capacity(state)
```

2.2.3 Inisialisasi State dan Pembentukan Neighbor

Tabel 6. Implementasi Inisialisasi dan Neighbor

Kategori	Nama	Deskripsi
Inisialisasi	generate_random_state	Mulai dari semua kendaraan outside; acak urutan dan coba tempatkan secara valid.
Peluang	inclusion_probability = 0.8	Peluang tiap kendaraan dicoba masuk saat inisialisasi.
Operator	move_vehicle	Pindahkan kendaraan ke koordinat lain atau muat kendaraan outside.
Operator	rotate_vehicle / swap_vehicles	Putar satu kendaraan atau tukar posisi dua kendaraan.
Tetangga	generate_neighbors	Enumerasi kandidat valid, buang duplikasi dengan state_key.

State awal dibangkitkan dengan random.Random(seed) sehingga susunan awal dapat diulang. Untuk satu kapal, generator mengacak urutan kendaraan dan mencoba koordinat/orientasi hingga menemukan penempatan valid atau mencapai 100 upaya per kendaraan.

Cuplikan kode: src/core/neighbor.py, baris 264-281

```
264  def generate_neighbors(
265      state: SearchState,
266      objective_mode: str = "shipping_fee",
267  ) -> list[SearchState]:
268      neighbors: list[SearchState] = []
269      seen = {state_key(state)}
270      candidates = chain(
271          _move_candidates(state, objective_mode),
272          _rotate_candidates(state, objective_mode),
273          _swap_candidates(state, objective_mode),
274          _fleet_candidates(state, objective_mode),
275      )
276      for candidate in candidates:
277          key = state_key(candidate)
278          if key not in seen:
279              seen.add(key)
280              neighbors.append(candidate)
281      return neighbors
```

2.2.4 Fungsi Objektif Alternatif

Tabel 7. Implementasi Fungsi Objektif Alternatif

Kategori	Nama	Deskripsi
Mode	fee_per_area	f(s) = Σ shipping_fee(v) / (width(v) × length(v)) untuk kendaraan inside.
Mode	fee_with_eta	f(s) = Σ shipping_fee(v) / (1 + eta(v)) untuk kendaraan inside.
Variabel	shipping_fee, area, eta	Semua berasal dari atribut kendaraan yang sudah tersedia.
Method	OBJECTIVE_MODES	Memilih fungsi skor per kendaraan berdasarkan nama mode.
Return	calculate_objective	Jumlah skor kendaraan inside menurut mode yang dipilih.

Dua mode alternatif mengubah prioritas solusi: fee_per_area menekankan tarif per luas, sedangkan fee_with_eta membagi tarif dengan 1 + ETA. Keduanya menggunakan atribut yang sudah ada dan tidak boleh dibaca sebagai jumlah ShippingFee asli.

Cuplikan kode: src/core/objective.py, baris 12-24

```
12  def _fee_per_area(placement: Placement) -> float:
13      return placement.vehicle.shipping_fee / placement.vehicle.area
14
15
16  def _fee_with_eta(placement: Placement) -> float:
17      return placement.vehicle.shipping_fee / (1.0 + placement.vehicle.eta)
18
19
20  OBJECTIVE_MODES: dict[str, Callable[[Placement], float]] = {
21      "shipping_fee": _shipping_fee,
22      "fee_per_area": _fee_per_area,
23      "fee_with_eta": _fee_with_eta,
24  }
```

2.3 Implementasi Algoritma Local Search

2.3.1 First Improvement Hill Climbing

Tabel 8. Implementasi First Improvement Hill Climbing

Kategori	Nama	Deskripsi
Parameter	variant, max_iterations, seed	Memilih first_improvement, membatasi langkah, dan menjaga pilihan acak tetap dapat diulang.
Variabel	better, current	Daftar neighbor dengan objective lebih tinggi daripada state saat ini.
Method	_pick_neighbor	Mengambil better[0] atau None apabila tidak ada perbaikan.
Proses	better[0]	Mengikuti urutan generator neighbor; pencarian berhenti saat pilihan kosong.
Return	dict hasil	State awal/akhir, objective terbaik, history, iterasi, dan durasi.
Lainnya	is_valid_state; with_updated_metrics	Semua varian berbagi pemeriksaan state dan pemutakhiran skor melalui generator neighbor.

Varian first improvement memakai fungsi hill_climb yang sama. Pemilihannya ditentukan oleh cabang _pick_neighbor atau oleh _random_restart. Setiap iterasi menyimpan objective dalam history dan hanya state valid yang diterima sebagai neighbor.

Kotak kode 2.3.1. First Improvement Hill Climbing

Algoritma atau Source Code

src/algorithms/hill_climbing.py, baris 30-41

```
30      better = [
31          neighbor
32          for neighbor in neighbors
33          if neighbor.objective_value > current.objective_value
34      ]
35
36      if variant == "first_improvement":
37          return better[0] if better else None
38      if variant == "steepest":
39          return max(better, key=lambda state: state.objective_value, default=None)
40      if variant == "stochastic":
41          return generator.choice(better) if better else None
```

src/algorithms/hill_climbing.py, baris 64-83

```
64      initial = with_updated_metrics(initial_state, objective_mode)
65      current = initial
66      history = [{"iteration": 0, "objective": current.objective_value}]
67      seen = {state_key(current)}
68      sideways_used = 0
69      iterations = 0
70
71      while iterations < max_iterations:
72          neighbors = generate_neighbors(current, objective_mode)
73          selected = _pick_neighbor(
74              current,
75              neighbors,
76              variant,
77              generator,
78              sideways_used,
79              max_sideways,
80              seen,
81          )
82          if selected is None:
83              break
```

src/algorithms/hill_climbing.py, baris 90-100

```
90          current = selected
91          seen.add(state_key(current))
92          iterations += 1
93          history.append({"iteration": iterations, "objective": current.objective_value})
94
95      return {
96          "initial_state": initial,
97          "final_state": current,
98          "best_objective": current.objective_value,
99          "iterations": iterations,
100         "history": history,
```

2.3.2 Steepest Ascent Hill Climbing

Tabel 9. Implementasi Steepest Ascent Hill Climbing

Kategori	Nama	Deskripsi
Parameter	variant, max_iterations, seed	Memilih steepest dan membatasi langkah pencarian.
Variabel	better, objective_value	Kumpulan neighbor yang meningkatkan nilai objective.
Method	_pick_neighbor	Memakai max pada better menurut objective_value.
Proses	argmax better	Mengevaluasi seluruh neighbor valid sebelum memilih kenaikan terbesar.
Return	dict hasil	State akhir, skor, jumlah iterasi, history, dan durasi.
Lainnya	is_valid_state; with_updated_metrics	Semua varian berbagi pemeriksaan state dan pemutakhiran skor melalui generator neighbor.

Varian steepest memakai fungsi hill_climb yang sama. Pemilihannya ditentukan oleh cabang _pick_neighbor atau oleh _random_restart. Setiap iterasi menyimpan objective dalam history dan hanya state valid yang diterima sebagai neighbor.

Kotak kode 2.3.2. Steepest Ascent Hill Climbing

Algoritma atau Source Code

src/algorithms/hill_climbing.py, baris 30-41

```
30      better = [
31          neighbor
32          for neighbor in neighbors
33          if neighbor.objective_value > current.objective_value
34      ]
35
36      if variant == "first_improvement":
37          return better[0] if better else None
38      if variant == "steepest":
39          return max(better, key=lambda state: state.objective_value, default=None)
40      if variant == "stochastic":
41          return generator.choice(better) if better else None
```

src/algorithms/hill_climbing.py, baris 71-93

```
71      while iterations < max_iterations:
72          neighbors = generate_neighbors(current, objective_mode)
73          selected = _pick_neighbor(
74              current,
75              neighbors,
76              variant,
77              generator,
78              sideways_used,
79              max_sideways,
80              seen,
81          )
82          if selected is None:
83              break
84
85          if selected.objective_value == current.objective_value:
86              sideways_used += 1
87          else:
88              sideways_used = 0
89
90          current = selected
91          seen.add(state_key(current))
92          iterations += 1
93          history.append({"iteration": iterations, "objective": current.objective_value})
```

2.3.3 Stochastic Hill Climbing

Tabel 10. Implementasi Stochastic Hill Climbing

Kategori	Nama	Deskripsi
Parameter	variant, max_iterations, seed	Memilih stochastic dengan sumber acak yang dapat diulang.
Variabel	better, generator	Daftar perbaikan dan random.Random untuk pengambilan sampel.
Method	_pick_neighbor	Memakai generator.choice(better) jika kandidat tersedia.
Proses	pilihan acak	Setiap neighbor yang lebih baik memiliki peluang pilihan sama; tanpa perbaikan berhenti.
Return	dict hasil	State akhir, skor, jumlah iterasi, history, dan durasi.
Lainnya	is_valid_state; with_updated_metrics	Semua varian berbagi pemeriksaan state dan pemutakhiran skor melalui generator neighbor.

Varian stochastic memakai fungsi hill_climb yang sama. Pemilihannya ditentukan oleh cabang _pick_neighbor atau oleh _random_restart. Setiap iterasi menyimpan objective dalam history dan hanya state valid yang diterima sebagai neighbor.

Kotak kode 2.3.3. Stochastic Hill Climbing

Algoritma atau Source Code

src/algorithms/hill_climbing.py, baris 30-41

```
30      better = [
31          neighbor
32          for neighbor in neighbors
33          if neighbor.objective_value > current.objective_value
34      ]
35
36      if variant == "first_improvement":
37          return better[0] if better else None
38      if variant == "steepest":
39          return max(better, key=lambda state: state.objective_value, default=None)
40      if variant == "stochastic":
41          return generator.choice(better) if better else None
```

src/algorithms/hill_climbing.py, baris 71-93

```
71      while iterations < max_iterations:
72          neighbors = generate_neighbors(current, objective_mode)
73          selected = _pick_neighbor(
74              current,
75              neighbors,
76              variant,
77              generator,
78              sideways_used,
79              max_sideways,
80              seen,
81          )
82          if selected is None:
83              break
84
85          if selected.objective_value == current.objective_value:
86              sideways_used += 1
87          else:
88              sideways_used = 0
89
90          current = selected
91          seen.add(state_key(current))
92          iterations += 1
93          history.append({"iteration": iterations, "objective": current.objective_value})
```

2.3.4 Hill Climbing with Sideways Move

Tabel 11. Implementasi Hill Climbing with Sideways Move

Kategori	Nama	Deskripsi
Parameter	max_sideways, max_iterations, seed	Membatasi langkah setara berturut-turut dan total iterasi tiap run.
Variabel	seen, sideways_used	Mencegah kembali ke state yang sudah dilihat dan mencatat langkah setara sejak kenaikan terakhir.
Method	_pick_neighbor	Memilih perbaikan terbaik, lalu kandidat setara yang belum seen jika jatah tersedia.
Proses	reset saat naik	sideways_used direset menjadi nol ketika objective meningkat.
Return	sideways_moves, maximum_sideways	Metadata jumlah langkah setara dan batas yang dipakai.
Lainnya	is_valid_state; with_updated_metrics	Semua varian berbagi pemeriksaan state dan pemutakhiran skor melalui generator neighbor.

Varian sideways memakai fungsi hill_climb yang sama. Pemilihannya ditentukan oleh cabang _pick_neighbor atau oleh _random_restart. Setiap iterasi menyimpan objective dalam history dan hanya state valid yang diterima sebagai neighbor.

Kotak kode 2.3.4. Hill Climbing with Sideways Move

Algoritma atau Source Code

src/algorithms/hill_climbing.py, baris 43-53

```
43      if better:
44          return max(better, key=lambda state: state.objective_value)
45      if sideways_used >= max_sideways:
46          return None
47      sideways = [
48          neighbor
49          for neighbor in neighbors
50          if neighbor.objective_value == current.objective_value
51          and state_key(neighbor) not in seen
52      ]
53      return generator.choice(sideways) if sideways else None
```

src/algorithms/hill_climbing.py, baris 64-93

```
64      initial = with_updated_metrics(initial_state, objective_mode)
65      current = initial
66      history = [{"iteration": 0, "objective": current.objective_value}]
67      seen = {state_key(current)}
68      sideways_used = 0
69      iterations = 0
70
71      while iterations < max_iterations:
72          neighbors = generate_neighbors(current, objective_mode)
73          selected = _pick_neighbor(
74              current,
75              neighbors,
76              variant,
77              generator,
78              sideways_used,
79              max_sideways,
80              seen,
81          )
82          if selected is None:
83              break
84
85          if selected.objective_value == current.objective_value:
86              sideways_used += 1
87          else:
88              sideways_used = 0
89
90          current = selected
91          seen.add(state_key(current))
92          iterations += 1
93          history.append({"iteration": iterations, "objective": current.objective_value})
```

src/algorithms/hill_climbing.py, baris 101-104

```
101         "sideways_moves": sum(
102             first["objective"] == second["objective"]
103             for first, second in zip(history, history[1:])
104         ),
```

2.3.5 Random Restart Hill Climbing

Tabel 12. Implementasi Random Restart Hill Climbing

Kategori	Nama	Deskripsi
Parameter	max_restart, max_iterations, seed	Tiga restart berarti satu run awal dan tiga run acak tambahan.
Variabel	runs, best_run, history	Menyimpan hasil setiap pendakian, mengambil skor terbesar, dan menggabungkan jejak.
Method	_random_restart	Memanggil _single_hill_climb dengan strategi steepest pada tiap awal baru.
Proses	generate_random_state	Membangkitkan state valid yang baru sebelum pendakian berikutnya.

Kategori	Nama	Deskripsi
Return	iterations_per_restart, restarts	Daftar panjang tiap pendakian dan jumlah restart tambahan.
Lainnya	is_valid_state; with_updated_metrics	Semua varian berbagi pemeriksaan state dan pemutakhiran skor melalui generator neighbor.

Varian random restart memakai fungsi hill_climb yang sama. Pemilihannya ditentukan oleh cabang _pick_neighbor atau oleh _random_restart. Setiap iterasi menyimpan objective dalam history dan hanya state valid yang diterima sebagai neighbor.

Kotak kode 2.3.5. Random Restart Hill Climbing

Algoritma atau Source Code

src/algorithms/hill_climbing.py, baris 108-144

```
108  def _random_restart(
109      initial_state: SearchState,
110      objective_mode: str,
111      max_iterations: int,
112      max_restart: int,
113      generator: random.Random,
114  ) -> dict[str, Any]:
115      runs = [
116          _single_hill_climb(
117              initial_state,
118              "steepest",
119              objective_mode,
120              max_iterations,
121              0,
122              generator,
123          )
124      ]
125
126      for _ in range(max_restart):
127          restart_state = generate_random_state(
128              ship_configuration(initial_state),
129              initial_state.vehicles,
130              objective_mode=objective_mode,
131              rng=generator,
132          )
133          runs.append(
134              _single_hill_climb(
135                  restart_state,
136                  "steepest",
137                  objective_mode,
138                  max_iterations,
139                  0,
140                  generator,
141              )
142          )
143
144      best_run = max(runs, key=lambda run: run["best_objective"])
```

src/algorithms/hill_climbing.py, baris 158-166

```
158      return {
159          "initial_state": initial_state,
160          "final_state": best_run["final_state"],
161          "best_objective": best_run["best_objective"],
162          "iterations": sum(run["iterations"] for run in runs),
163          "history": history,
164          "restarts": max_restart,
165          "iterations_per_restart": [run["iterations"] for run in runs],
166          "maximum_restart": max_restart,
```

2.3.6 Simulated Annealing

Tabel 13. Implementasi Simulated Annealing

Kategori	Nama	Deskripsi
Parameter	T0, cooling_rate, Tmin	Suhu awal, pengali pendinginan, dan ambang suhu untuk menghentikan loop.

Kategori	Nama	Deskripsi
Variabel	current, best, delta	Kandidat dibandingkan terhadap state berjalan; solusi terbaik disimpan terpisah.
Method	random_neighbor	Mengambil tetangga valid secara acak dengan batas neighbor_attempts.
Proses	Penerimaan dan pendinginan	Delta >= 0 diterima; bila delta < 0 diterima dengan exp(delta/T). Setiap iterasi T dikali cooling_rate.
Return	final_state, history	Mengembalikan state terbaik beserta objective, suhu, probabilitas, durasi, dan catatan stuck.
Lainnya	stuck_threshold	stuck_count bertambah tiap 25 iterasi tanpa perbaikan best pada konfigurasi eksperimen.

Simulated annealing memulai current dan best dari state valid, lalu mengambil satu random_neighbor per iterasi. Kandidat dengan selisih skor $\Delta \geq 0$ diterima dengan penurunan diterima bila bilangan acak lebih kecil dari $\exp(\Delta/T)$. Pencarian berhenti pada batas iterasi, suhu minimum, atau ketika tidak ada tetangga yang berhasil dibuat. final_state adalah best yang pernah ditemukan, bukan selalu current pada iterasi terakhir.

Kotak kode 2.3.6. Simulated Annealing

Algoritma atau Source Code

src/algorithms/simulated_annealing.py, baris 12–22

```
12  def simulated_annealing(
13      state: SearchState,
14      initial_temperature: float = 100.0,
15      cooling_rate: float = 0.995,
16      minimum_temperature: float = 0.01,
17      max_iterations: int = 1000,
18      neighbor_attempts: int = 100,
19      stuck_threshold: int = 25,
20      objective_mode: str = "shipping_fee",
21      seed: int | None = None,
22  ) -> dict[str, Any]:
```

src/algorithms/simulated_annealing.py, baris 53–93

```
53      iteration = 0
54      while iteration < max_iterations and temperature > minimum_temperature:
55          candidate = random_neighbor(
56              current,
57              generator,
58              objective_mode,
59              neighbor_attempts,
60          )
61          if candidate is None:
62              break
63
64          delta = candidate.objective_value - current.objective_value
65          acceptance_probability = 1.0 if delta >= 0 else math.exp(delta / temperature)
66          accepted = delta >= 0 or generator.random() < acceptance_probability
67          if accepted:
68              current = candidate
69              accepted_moves += 1
70
71          if current.objective_value > best.objective_value:
72              best = current
73              iterations_without_improvement = 0
74          else:
75              iterations_without_improvement += 1
76
77          stuck = iterations_without_improvement == stuck_threshold
78          if stuck:
79              stuck_count += 1
80              iterations_without_improvement = 0
81
82          iteration += 1
83          temperature *= cooling_rate
84          history.append(
85              {
86                  "iteration": iteration,
87                  "objective": current.objective_value,
```

Algoritma atau Source Code

```
                "best_objective": best.objective_value,
                "temperature": temperature,
                "acceptance_probability": acceptance_probability,
                "accepted": accepted,
                "stuck": stuck,
            }
```

src/algorithms/simulated_annealing.py, baris 96–109

```
    return {
        "initial_state": initial,
        "final_state": best,
        "best_objective": best.objective_value,
        "iterations": iteration,
        "duration": perf_counter() - started_at,
        "history": history,
        "accepted_moves": accepted_moves,
        "stuck_count": stuck_count,
        "initial_temperature": initial_temperature,
        "final_temperature": temperature,
        "cooling_rate": cooling_rate,
        "objective_mode": objective_mode,
    }
```

2.3.7 Genetic Algorithm

Tabel 14. Implementasi Genetic Algorithm

Kategori	Nama	Deskripsi
Parameter	population_size, generations	Jumlah individu per generasi dan banyak siklus reproduksi; kedua variabel diuji terpisah.
Variabel	population, best, history	Populasi awal mencakup state yang diberikan; best menyimpan solusi terbaik lintas generasi.
Method	_select_parent	Tournament: sampel sejumlah tournament_size, lalu ambil objective tertinggi.
Proses	_crossover, _mutate	Crossover satu titik pada daftar Placement; child dirakit sambil memeriksa validitas; mutasi memakai random_neighbor.
Return	final_state, history	Skor terbaik dan max/average/min fitness untuk generasi 0 hingga generasi terakhir.
Lainnya	elitism_count	Sebanyak elitism_count individu terbaik disalin langsung ke generasi berikutnya.

Gen direpresentasikan sebagai Placement kendaraan. Populasi berisi state awal dan state valid lain hasil generate_random_state. Tournament memilih orang tua crossover satu titik menyusun dua child, sementara _build_valid_child hanya menerima penempatan yang tidak melanggar constraint (khusus FleetState dapat mencoba menempatkan ulang). Mutasi per individu terjadi dengan mutation_rate dan menggunakan random_neighbor. Elitisme mempertahankan dua individu terbaik pada konfigurasi eksperimen.

Kotak kode 2.3.7. Genetic Algorithm

Algoritma atau Source Code

src/algorithms/genetic_algorithm.py, baris 13–28

```
 13  def _build_valid_child(
 14      parent: SearchState,
 15      genes: list[Placement],
 16      objective_mode: str,
 17      generator: random.Random,
 18  ) -> SearchState:
 19      child = create_outside_state(ship_configuration(parent), parent.vehicles, objective_mode)
 20      for index, gene in enumerate(genes):
 21          if not gene.inside:
 22              continue
 23          candidate = child.replace_placement(index, gene)
 24          if is_valid_state(candidate):
 25              child = candidate
 26          elif isinstance(child, FleetState):
 27              child = place_randomly(child, index, generator, objective_mode, max_attempts=20)
 28      return with_updated_metrics(child, objective_mode)
```

src/algorithms/genetic_algorithm.py, baris 31–49

```
 31  def _crossover(
 32      first: SearchState,
 33      second: SearchState,
 34      generator: random.Random,
 35      crossover_rate: float,
 36      objective_mode: str,
 37  ) -> tuple[SearchState, SearchState]:
 38      if len(first.placements) < 2 or generator.random() >= crossover_rate:
 39          return (
 40              with_updated_metrics(first, objective_mode),
 41              with_updated_metrics(second, objective_mode),
 42          )
 43      point = generator.randrange(1, len(first.placements))
 44      first_genes = first.placements[:point] + second.placements[point:]
 45      second_genes = second.placements[:point] + first.placements[point:]
 46      return (
 47          _build_valid_child(first, first_genes, objective_mode, generator),
 48          _build_valid_child(second, second_genes, objective_mode, generator),
 49      )
```

src/algorithms/genetic_algorithm.py, baris 52–71

```
 52  def _mutate(
 53      state: SearchState,
 54      generator: random.Random,
 55      mutation_rate: float,
 56      mutation_attempts: int,
 57      objective_mode: str,
 58  ) -> SearchState:
 59      if generator.random() >= mutation_rate:
 60          return state
 61      candidate = random_neighbor(state, generator, objective_mode, mutation_attempts)
 62      return candidate if candidate is not None else state
 63
 64
 65  def _select_parent(
 66      population: list[SearchState],
 67      tournament_size: int,
 68      generator: random.Random,
 69  ) -> SearchState:
 70      candidates = generator.sample(population, tournament_size)
 71      return max(candidates, key=lambda state: state.objective_value)
```

src/algorithms/genetic_algorithm.py, baris 114–129

```
114      generator = random.Random(seed)
115      initial = with_updated_metrics(state, objective_mode)
116      population = [initial]
117      for _ in range(population_size - 1):
118          population.append(
119              generate_random_state(
120                  ship_configuration(state),
121                  state.vehicles,
122                  objective_mode=objective_mode,
123                  rng=generator,
124              )
125          )
126
127      best = max(population, key=lambda candidate: candidate.objective_value)
128      history = [_generation_summary(population, 0)]
129      started_at = perf_counter()
```

src/algorithms/genetic_algorithm.py, baris 131–168

Algoritma atau Source Code

```
    for generation in range(1, generations + 1):
        ranked = sorted(
            population,
            key=lambda candidate: candidate.objective_value,
            reverse=True,
        )
        next_population = ranked[:elitism_count]

        while len(next_population) < population_size:
            first_parent = _select_parent(population, tournament_size, generator)
            second_parent = _select_parent(population, tournament_size, generator)
            children = _crossover(
                first_parent,
                second_parent,
                generator,
                crossover_rate,
                objective_mode,
            )
            for child in children:
                mutated = _mutate(
                    child,
                    generator,
                    mutation_rate,
                    mutation_attempts,
                    objective_mode,
                )
                next_population.append(mutated)
                if len(next_population) == population_size:
                    break

        population = next_population
        generation_best = max(
            population,
            key=lambda candidate: candidate.objective_value,
        )
        if generation_best.objective_value > best.objective_value:
            best = generation_best
        history.append(_generation_summary(population, generation))
```

2.3.8 Penanganan Multi-Kapal

Tabel 15. Implementasi Penanganan Multi-Kapal

Kategori	Nama	Deskripsi
Parameter	ships, ship_id	Input multi-kapal berupa dictionary ID kapal ke Ship; Placement inside menunjuk ID kapal tujuan.
Variabel	FleetState.placements	Seluruh kendaraan muncul tepat satu kali, termasuk yang outside; total objective dihitung untuk seluruh armada.
Method	ship_state, ship_summaries	Memproyeksikan isi sebuah kapal dan melaporkan berat, kapasitas, luas, shipping fee, serta validitas per kapal.
Proses	transfer_and_load	Memindahkan kendaraan dari sebuah kapal ke kapal lain sambil mengisi tempatnya dengan kendaraan outside, jika hasilnya valid.
Return	FleetState / summaries	State akhir menyimpan penugasan kapal, koordinat, orientasi, dan metrik armada.
Lainnya	is_valid_state	Batas dek, overlap dan kapasitas diperiksa per kapal; assignment ID harus tersedia.

Pada input ships, load_problem membuat map kapal dengan ID unik, lalu generate_random_state membentuk FleetState. Setiap kendaraan inside memiliki ship_id yang valid dan kendaraan outside tidak memiliki ship_id atau koordinat. Operasi pindah atau tukar hanya menghasilkan tetangga apabila is_valid_state terpenuhi. ship_summaries menghitung nilai dan pemakaian kapasitas tiap kapal, sehingga objective armada merupakan jumlah skor kendaraan yang dimuat di seluruh kapal.

Kotak kode 2.3.8. Penanganan Multi-Kapal

Algoritma atau Source Code

src/models/fleet_state.py, baris 28–49

```
            raise ValueError("Invalid fleet placement assignment")

    def has_valid_assignments(self) -> bool:
        for placement in self.placements:
            if type(placement.inside) is not bool or not isinstance(placement.orientation, Orientation):
                return False
            if placement.inside:
                if not isinstance(placement.ship_id, str) or placement.ship_id not in self.ships:
                    return False
                if type(placement.x) is not int or type(placement.y) is not int:
                    return False
            elif placement.ship_id is not None or placement.x is not None or placement.y is not None:
                return False
        return True

    @property
    def inside_placements(self) -> list[Placement]:
        return [placement for placement in self.placements if placement.inside]

    @property
    def outside_placements(self) -> list[Placement]:
        return [placement for placement in self.placements if not placement.inside]
```

src/core/neighbor.py, baris 196–223

```
def transfer_and_load(
    state: FleetState,
    vehicle_id: str,
    outside_vehicle_id: str,
    ship_id: str,
    x: int,
    y: int,
    orientation: Orientation | str | None = None,
    loading_orientation: Orientation | str | None = None,
    objective_mode: str = "shipping_fee",
) -> FleetState | None:
    first_index = _find_index(state, vehicle_id)
    second_index = _find_index(state, outside_vehicle_id)
    first = state.placements[first_index]
    second = state.placements[second_index]
    if (not first.inside or second.inside or first.ship_id == ship_id
            or ship_id not in state.ships):
        return None
    placements = state.placements.copy()
    placements[first_index] = replace(
        first, ship_id=ship_id, x=x, y=y, orientation=orientation or first.orientation,
    )
    placements[second_index] = replace(
        second, inside=True, ship_id=first.ship_id, x=first.x, y=first.y,
        orientation=loading_orientation or first.orientation,
    )
    candidate = state.replace_placements(placements)
    return with_updated_metrics(candidate, objective_mode) if is_valid_state(candidate) else None
```

src/core/objective.py, baris 48–70

```
def ship_summaries(state: FleetState, mode: str = "shipping_fee") -> list[dict]:
    from src.core.constraints import is_valid_state

    summaries = []
    for ship_id, ship in state.ships.items():
        local = state.ship_state(ship_id)
        area = sum(placement.vehicle.area for placement in local.inside_placements)
        weight = calculate_total_weight(local)
        summaries.append({
            "ship_id": ship_id,
            "loaded_ids": [placement.vehicle.id for placement in local.inside_placements],
            "loaded_count": len(local.inside_placements),
            "weight": weight,
            "capacity": ship.max_capacity,
            "weight_utilization": weight / ship.max_capacity if ship.max_capacity else 0.0,
            "occupied_area": area,
            "deck_area": ship.width * ship.length,
            "area_utilization": area / (ship.width * ship.length),
            "shipping_fee": calculate_objective(local),
            "objective": calculate_objective(local, mode),
            "valid": is_valid_state(local),
        })
    return summaries
```

2.3.9 Program Utama, Runner Eksperimen, dan Visualisasi

Tabel 16. Implementasi Program Utama, Runner Eksperimen, dan Visualisasi

Kategori	Nama	Deskripsi
Parameter	--input, --algorithm, --seed	CLI menerima path JSON, HC/SA/GA, varian HC, mode objective, dan parameter pencarian.
Variabel	args, state, result	Parser membaca opsi, inisialisasi state memakai random.Random(seed), lalu memilih fungsi algoritma.
Method	main, run_experiments	Run tunggal memanggil algoritma dan export_result; runner mengulang eksperimen dari config.
Proses	load_problem → init → search	Input divalidasi; state sah dibuat; hasil diberi metadata dan diekspor ke folder kosong.
Return	result.json, CSV, PNG	State awal/akhir, history, grafik objective atau fitness, grafik peluang SA, dan ringkasan kapal disimpan.
Lainnya	--render-experiments	JSON eksperimen yang sudah ada dapat diekspor ulang menjadi tabel dan grafik.

Program utama membangun parser perintah, memuat input, memilih algoritma, menginisialisasi state dari seed, kemudian mengekspor hasil dan mencetak metrik. Runner terpisah membaca konfigurasi, mengulang run, serta memvariasikan generasi dan populasi GA. export_result menolak state tidak valid sebelum menulis gambar state, grafik, CSV history, dan result.json.

Kotak kode 2.3.9. Program Utama, Runner Eksperimen, dan Visualisasi

Algoritma atau Source Code

src/main.py, baris 112–143

```
112             if algorithm == "hc":
113                 params.update(variant=args.variant, max_iterations=args.max_iterations,
114                               max_sideways=args.max_sideways, max_restart=args.max_restart)
115             elif algorithm == "sa":
116                 params.update(max_iterations=args.max_iterations, initial_temperature=args.temperature,
117                               cooling_rate=args.cooling_rate)
118             else:
119                 params.update(population_size=args.population, generations=args.generations)
120             if args.params:
121                 with args.params.open(encoding="utf-8") as stream:
122                     extra = json.load(stream)
123                 if not isinstance(extra, dict):
124                     raise ValueError("File params harus berisi JSON object")
125                 params.update(extra)
126             state = generate_random_state(ship, vehicles, objective_mode=params["objective_mode"],
127                                           rng=random.Random(params["seed"]))
128             function = {"hc": hill_climb, "sa": simulated_annealing, "ga": genetic_algorithm}[algorithm]
129             result = function(state, **params)
130
131             result.update(algorithm=algorithm, seed=params["seed"], parameters=params,
132                           input_file=str(args.input.resolve()))
133             export_result(result, destination, algorithm)
134             print_result(result, algorithm)
135         print(f"\nFolder output: {destination.resolve()}")
136         return 0
137     except (OSError, ValueError, TypeError, KeyError, EOFError) as error:
138         print(f"Gagal: {error}", file=sys.stderr)
139         return 1
140
141
142 if __name__ == "__main__":
143     raise SystemExit(main())
```

src/experiments/runner.py, baris 148–165

```
148 def run_experiments(
149     input_path: str | Path,
150     output_path: str | Path,
151     config_path: str | Path | None = None,
152 ) -> dict[str, Any]:
153     ship, vehicles = _load_problem(input_path)
154     config = load_experiment_config(config_path)
```

Algoritma atau Source Code

```
155      results = {
156          "input": {
157              **({"ships": [{"id": ship_id, **item.to_dict()} for ship_id, item in ship.items()]}
158                 if isinstance(ship, dict) else {"ship": ship.to_dict()}),
159              "vehicles": [vehicle.to_dict() for vehicle in vehicles],
160          },
161          "configuration": config,
162          "hill_climbing": _run_hill_climbing(ship, vehicles, config),
163          "simulated_annealing": _run_simulated_annealing(ship, vehicles, config),
164          "genetic_algorithm": _run_genetic_algorithm(ship, vehicles, config),
165      }
```

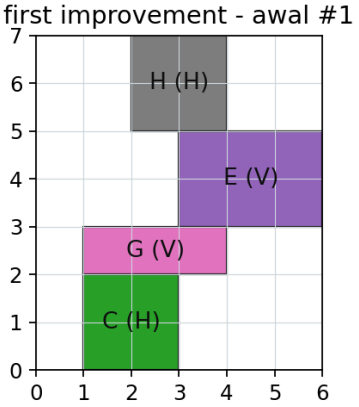
src/visualization/export.py, baris 53–86

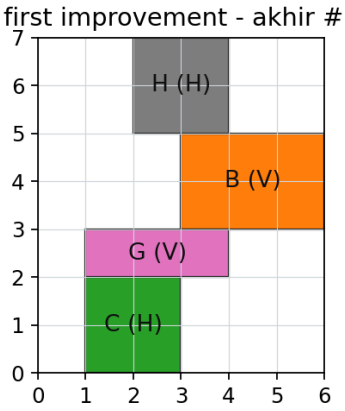
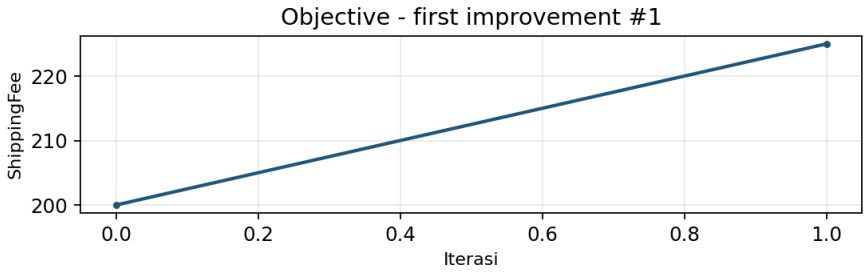
```
 53  def export_result(result, destination, algorithm):
 54      destination = Path(destination)
 55      destination.mkdir(parents=True, exist_ok=True)
 56      mode = result.get("objective_mode", "shipping_fee")
 57      result = dict(result)
 58      summaries = []
 59      for key, title in [("initial_state", "Initial state"), ("final_state", "Final / best state")]:
 60          state = result[key]
 61          if isinstance(state, dict):
 62              state = restore_state(state)
 63          if not is_valid_state(state):
 64              raise ValueError(f"{key} tidak valid; output dihentikan")
 65          if isinstance(state, FleetState):
 66              result.update(problem_type="multi_ship", ship_count=len(state.ships))
 67              result[f"{key}_ships"] = ship_summaries(state, mode)
 68              for summary in result[f"{key}_ships"]:
 69                  summaries.append({"stage": key, **summary, "loaded_ids": ", ".join(summary["loaded_ids"])})
 70          fig = draw_state(state, f"{algorithm.upper()} - {title}", destination / f"{key}.png", mode)
 71          plt.close(fig)
 72      history = result["history"]
 73      fleet = result.get("problem_type") == "multi_ship"
 74      if algorithm == "ga":
 75          plt.close(plot_ga_fitness(history, destination / "fitness_max_average_plot.png",
 76                                    title="Genetic Algorithm - fitness armada" if fleet else "Genetic Algorithm -
fitness populasi"))
 77      else:
 78          plt.close(plot_objective(history, destination / "objective_plot.png",
 79                                   title="Objective armada vs iterasi" if fleet else "Objective vs iterasi"))
 80      if algorithm == "sa":
 81          plt.close(plot_sa_probability(history, destination / "probability_plot.png"))
 82      write_csv(destination / "history.csv", history)
 83      if summaries:
 84          write_csv(destination / "ships.csv", summaries)
 85      with (destination / "result.json").open("w", encoding="utf-8") as stream:
 86          json.dump(serialize(result), stream, indent=2, allow_nan=False)
```

2.4 Hasil Eksperimen

2.4.1 Hasil First Improvement Hill Climbing

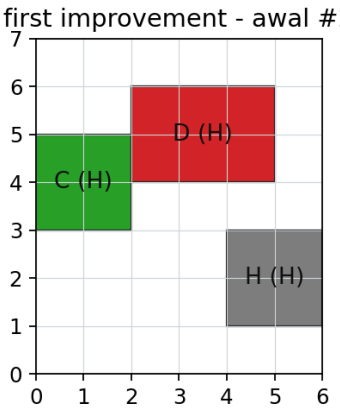
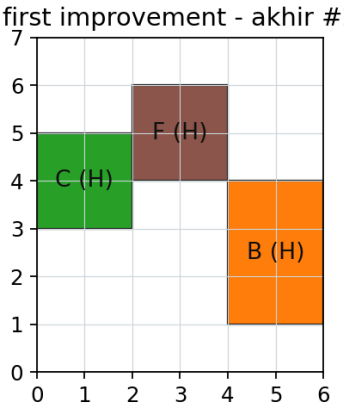
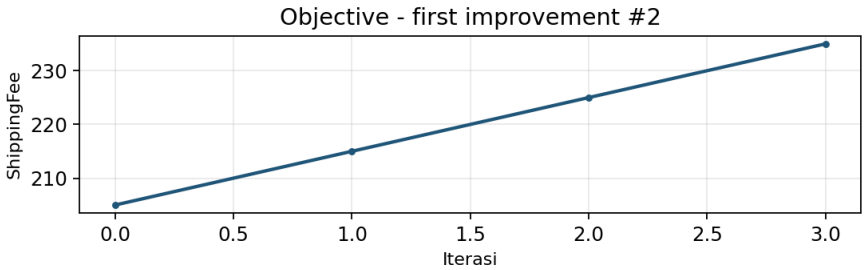
Tabel 17. Hasil First Improvement Hill Climbing , Percobaan 1

Percobaan 1	
State Awal	 **Muatan dan nilai** Inside: C, E, G, H Outside: A, B, D, F ShippingFee: 200 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah

Percobaan 1	
State Akhir	 **Muatan dan nilai** Inside: B, C, G, H Outside: A, D, E, F ShippingFee: 225 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	225 (awal 200); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0,0056 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	1 iterasi; seed 2027; max_iterations 100

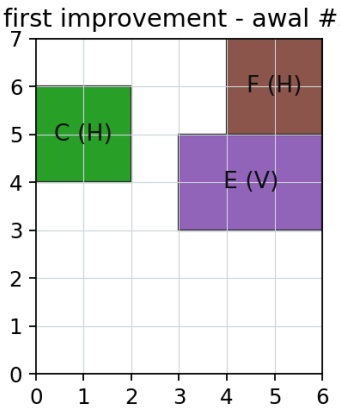
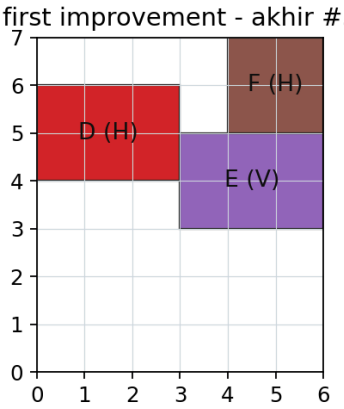
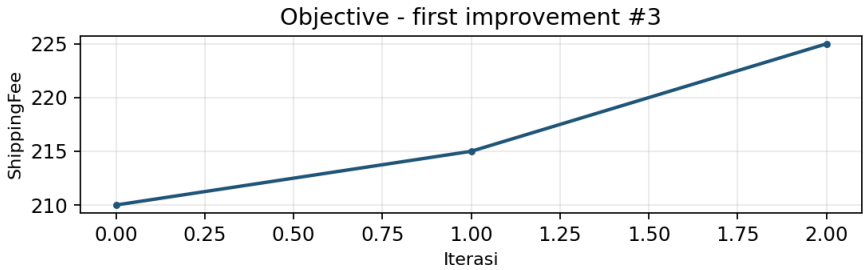
Percobaan 1: seed 2027 menaikkan objective 200 menjadi 225. Kendaraan akhir yang dimuat: B, C, G, H. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

Tabel 18. Hasil First Improvement Hill Climbing , Percobaan 2

Percobaan 2	
State Awal	 **Muatan dan nilai** Inside: C, D, H Outside: A, B, E, F, G ShippingFee: 205 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	235 (awal 205); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0122 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	3 iterasi; seed 2028; max_iterations 100

Percobaan 2: seed 2028 menaikkan objective 205 menjadi 235. Kendaraan akhir yang dimuat: B, C, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

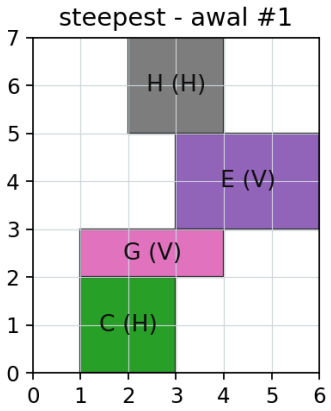
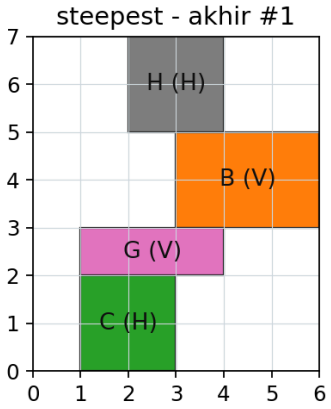
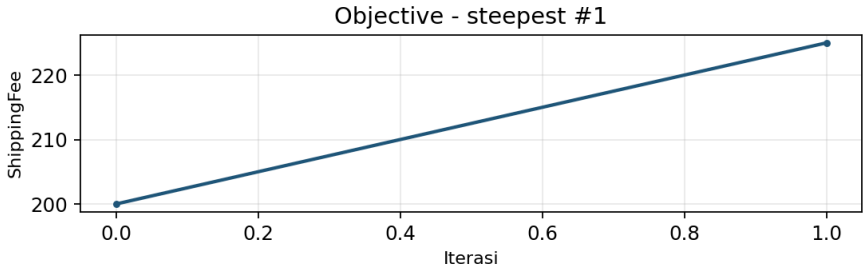
Tabel 19. Hasil First Improvement Hill Climbing , Percobaan 3

Percobaan 3	
State Awal	 **Muatan dan nilai** Inside: C, E, F Outside: A, B, D, G, H ShippingFee: 210 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: D, E, F Outside: A, B, C, G, H ShippingFee: 225 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	225 (awal 210); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0141 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	2 iterasi; seed 2029; max_iterations 100

Percobaan 3: seed 2029 menaikkan objective 210 menjadi 225. Kendaraan akhir yang dimuat: D, E, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

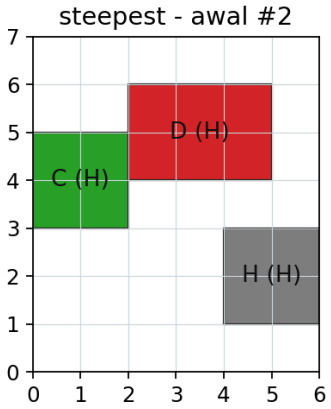
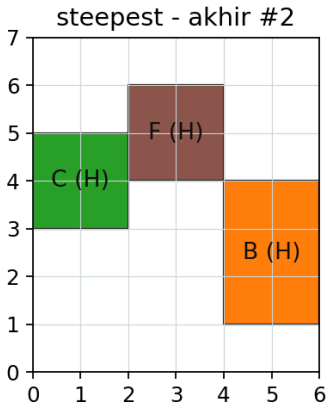
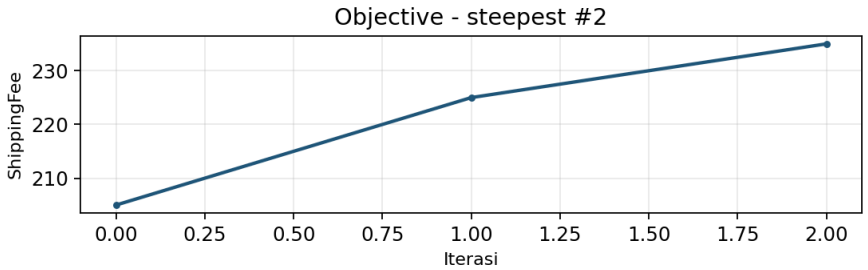
2.4.2 Hasil Steepest Ascent Hill Climbing

Tabel 20. Hasil Steepest Ascent Hill Climbing , Percobaan 1

Percobaan 1	
State Awal	 **Muatan dan nilai** Inside: C, E, G, H Outside: A, B, D, F ShippingFee: 200 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, G, H Outside: A, D, E, F ShippingFee: 225 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	225 (awal 200); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0069 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	1 iterasi; seed 2027; max_iterations 100

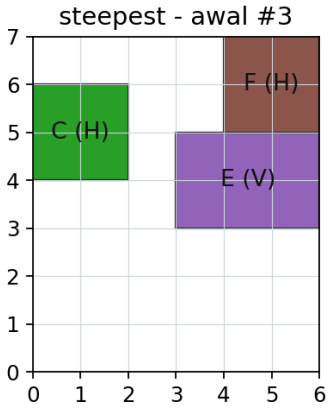
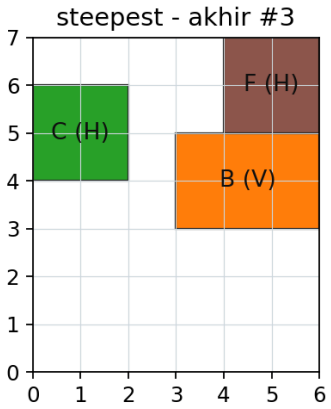
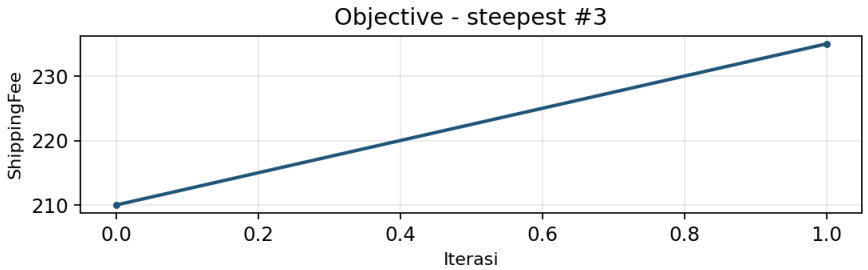
Percobaan 1: seed 2027 menaikkan objective 200 menjadi 225. Kendaraan akhir yang dimuat: B, C, G, H. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

Tabel 21. Hasil Steepest Ascent Hill Climbing , Percobaan 2

Percobaan 2	
State Awal	 **Muatan dan nilai** Inside: C, D, H Outside: A, B, E, F, G ShippingFee: 205 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	235 (awal 205); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0090 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	2 iterasi; seed 2028; max_iterations 100

Percobaan 2: seed 2028 menaikkan objective 205 menjadi 235 Kendaraan akhir yang dimuat: B, C, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

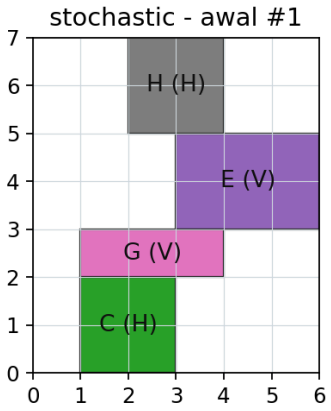
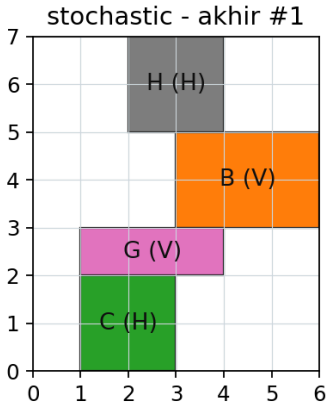
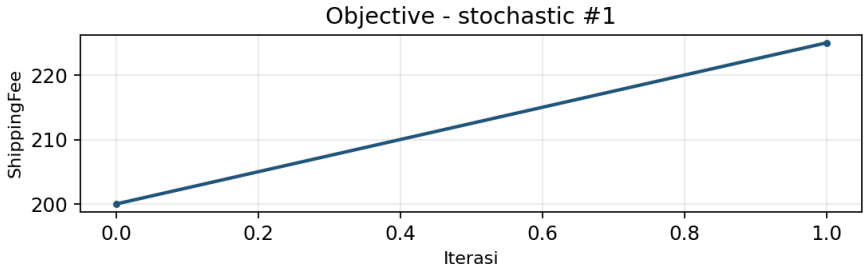
Tabel 22. Hasil Steepest Ascent Hill Climbing , Percobaan 3

Percobaan 3	
State Awal	 **Muatan dan nilai** Inside: C, E, F Outside: A, B, D, G, H ShippingFee: 210 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	235 (awal 210); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0078 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	1 iterasi; seed 2029; max_iterations 100

Percobaan 3: seed 2029 menaikkan objective 210 menjadi 235 Kendaraan akhir yang dimuat: B, C, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

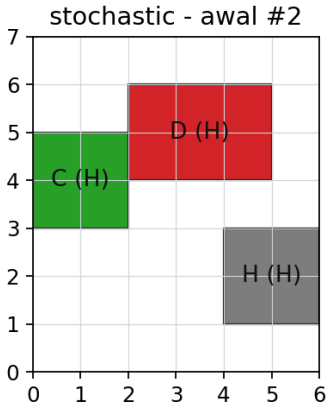
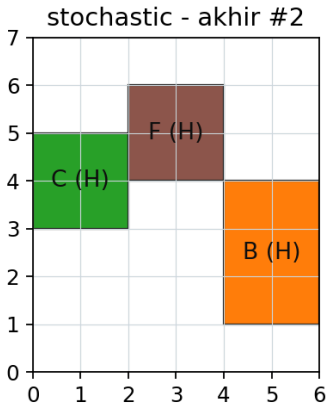
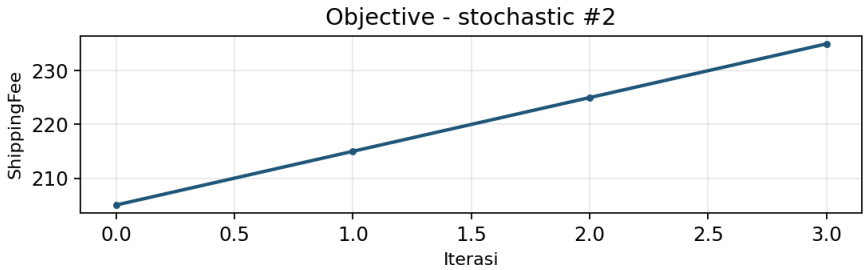
2.4.3 Hasil Stochastic Hill Climbing

Tabel 23. Hasil Stochastic Hill Climbing , Percobaan 1

Percobaan 1	
State Awal	 **Muatan dan nilai** Inside: C, E, G, H Outside: A, B, D, F ShippingFee: 200 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, G, H Outside: A, D, E, F ShippingFee: 225 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	225 (awal 200); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0080 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	1 iterasi; seed 2027; max_iterations 100

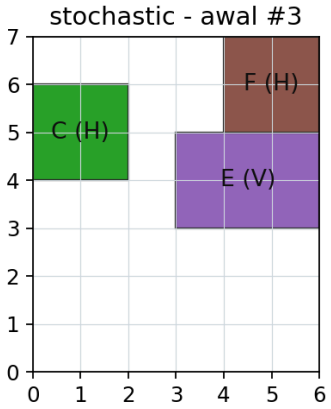
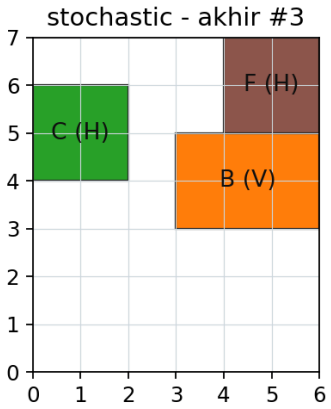
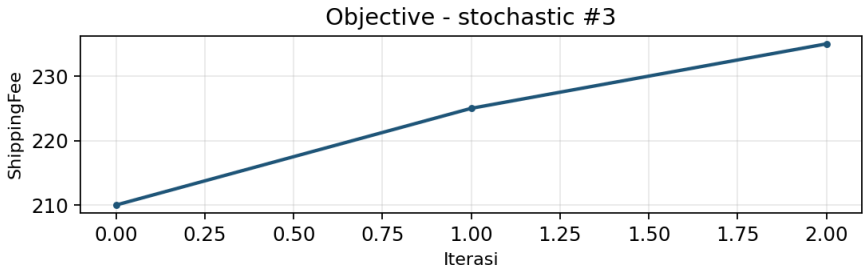
Percobaan 1: seed 2027 menaikkan objective 200 menjadi 225 Kendaraan akhir yang dimuat: B, C, G, H. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

Tabel 24. Hasil Stochastic Hill Climbing , Percobaan 2

Percobaan 2	
State Awal	 **Muatan dan nilai** Inside: C, D, H Outside: A, B, E, F, G ShippingFee: 205 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	235 (awal 205); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0182 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	3 iterasi; seed 2028; max_iterations 100

Percobaan 2: seed 2028 menaikkan objective 205 menjadi 235 Kendaraan akhir yang dimuat: B, C, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

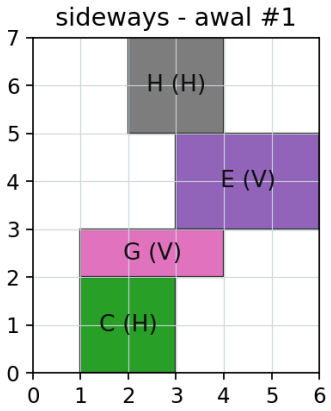
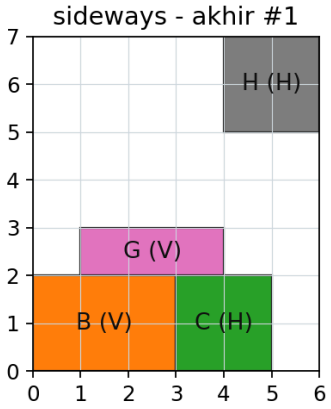
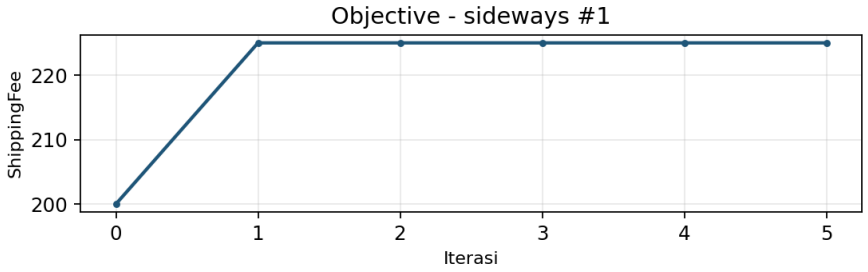
Tabel 25. Hasil Stochastic Hill Climbing , Percobaan 3

Percobaan 3	
State Awal	 **Muatan dan nilai** Inside: C, E, F Outside: A, B, D, G, H ShippingFee: 210 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	235 (awal 210); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0103 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	2 iterasi; seed 2029; max_iterations 100

Percobaan 3: seed 2029 menaikkan objective 210 menjadi 235 Kendaraan akhir yang dimuat: B, C, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

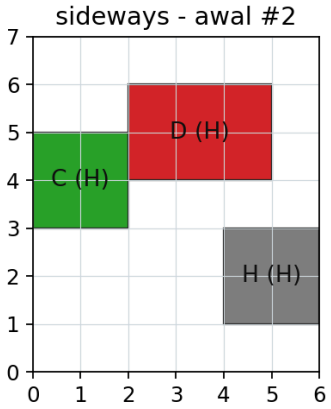
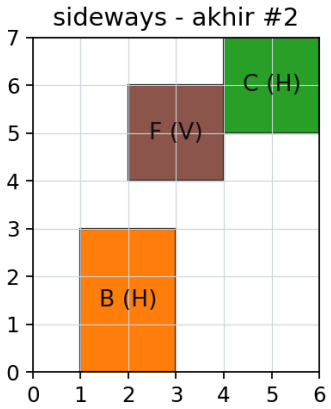
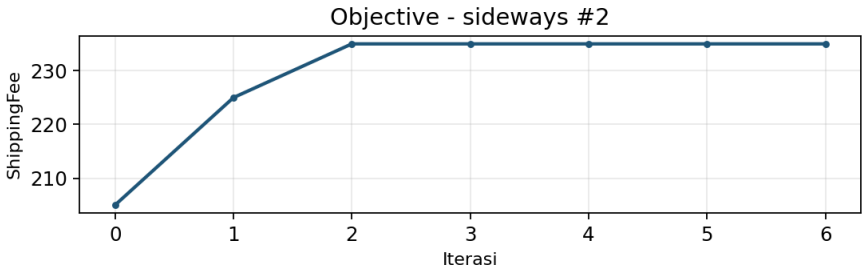
2.4.4 Hasil Hill Climbing with Sideways Move

Tabel 26. Hasil Hill Climbing with Sideways Move , Percobaan 1

Percobaan 1	
State Awal	 **Muatan dan nilai** Inside: C, E, G, H Outside: A, B, D, F ShippingFee: 200 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, G, H Outside: A, D, E, F ShippingFee: 225 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	225 (awal 200); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0212 detik ((diukur menggunakan time.perf_counter)
Iterasi dan Parameter	5 iterasi; seed 2027; max_iterations 100; max_sideways 4; sideways terjadi 4 kali

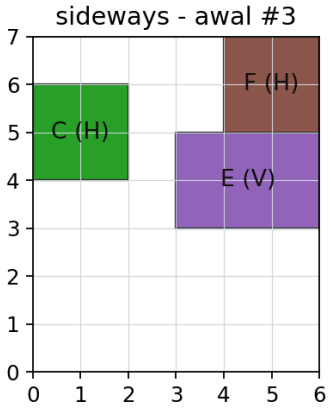
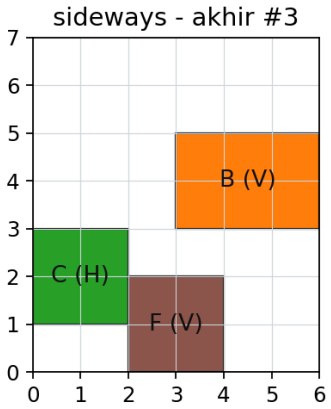
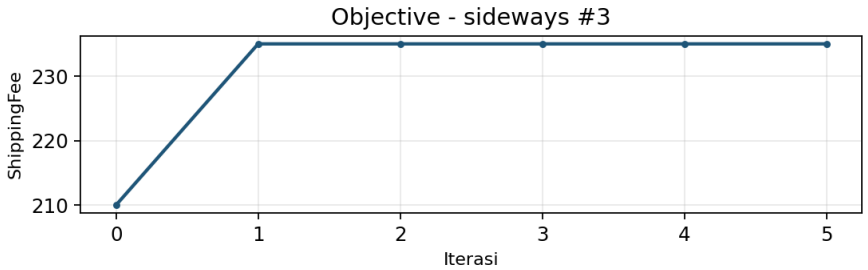
Percobaan 1: seed 2027 menaikkan objective 200 menjadi 225 Kendaraan akhir yang dimuat: B, C, G, H. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

Tabel 27. Hasil Hill Climbing with Sideways Move , Percobaan 2

Percobaan 2	
State Awal	 **Muatan dan nilai** Inside: C, D, H Outside: A, B, E, F, G ShippingFee: 205 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	235 (awal 205); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0232 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	6 iterasi; seed 2028; max_iterations 100; max_sideways 4; sideways terjadi 4 kali

Percobaan 2: seed 2028 menaikkan objective 205 menjadi 235. Kendaraan akhir yang dimuat: B, C, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

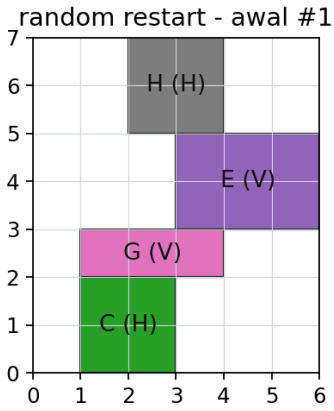
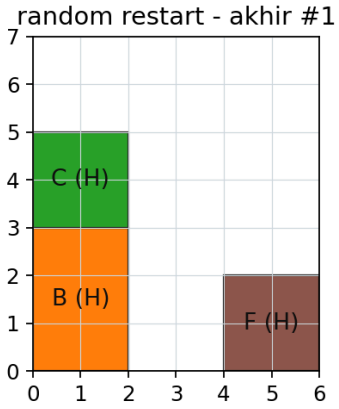
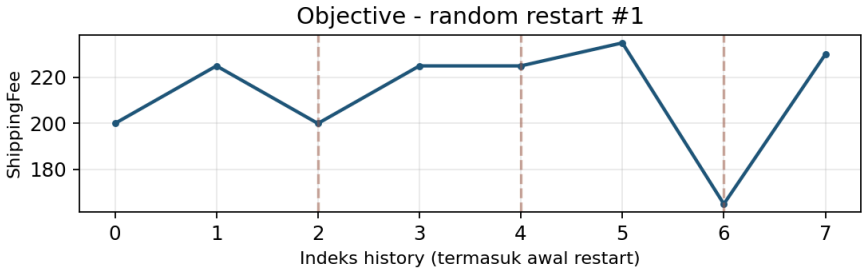
Tabel 28. Hasil Hill Climbing with Sideways Move , Percobaan 3

Percobaan 3	
State Awal	 **Muatan dan nilai** Inside: C, E, F Outside: A, B, D, G, H ShippingFee: 210 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	235 (awal 210); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0164 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	5 iterasi; seed 2029; max_iterations 100; max_sideways 4; sideways terjadi 4 kali

Percobaan 3: seed 2029 menaikkan objective 210 menjadi 235. Kendaraan akhir yang dimuat: B, C, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

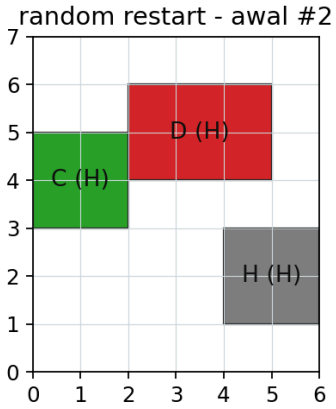
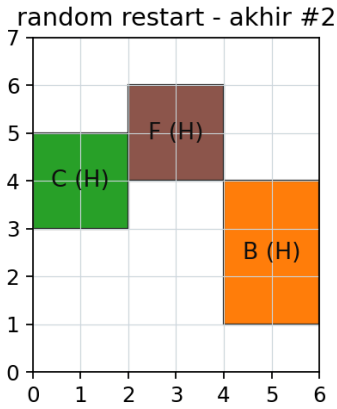
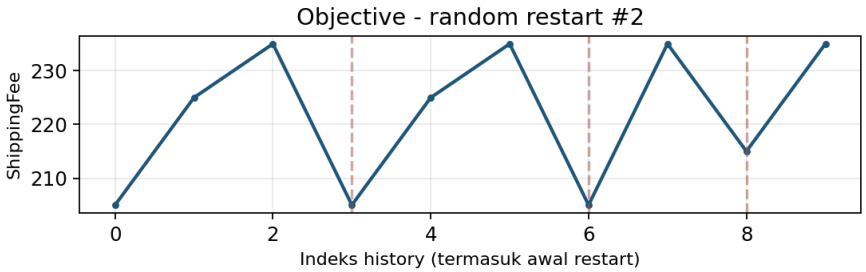
2.4.5 Hasil Random Restart Hill Climbing

Tabel 29. Hasil Random Restart Hill Climbing , Percobaan 1

Percobaan 1	
State Awal	 **Muatan dan nilai** Inside: C, E, G, H Outside: A, B, D, F ShippingFee: 200 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	235 (awal 200); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0282 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	4 iterasi; seed 2027; Batas 100 iterasi per pendakian; maksimum 3 restart tambahan, sehingga terdapat 4 pendakian.; max_restart 3; iterasi per run [1, 1, 1, 1]

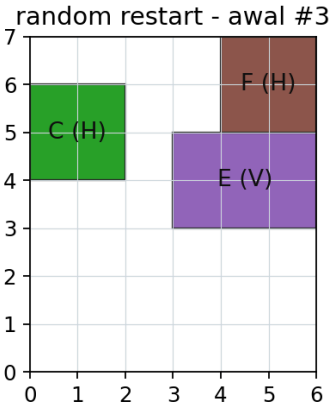
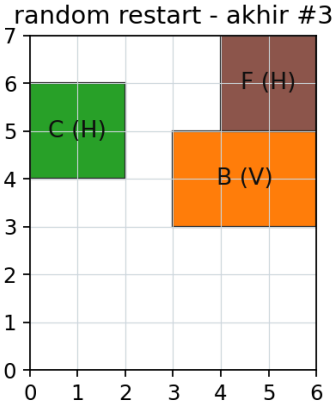
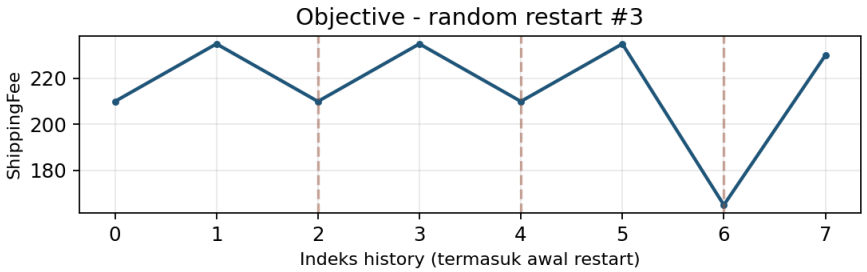
Percobaan 1: seed 2027 menaikkan objective 200 menjadi 235 Kendaraan akhir yang dimuat: B, C, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

Tabel 30. Hasil Random Restart Hill Climbing , Percobaan 2

Percobaan 2	
State Awal	 **Muatan dan nilai** Inside: C, D, H Outside: A, B, E, F, G ShippingFee: 205 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	235 (awal 205); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0310 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	6 iterasi; seed 2028; Batas 100 iterasi per pendakian; maksimum 3 restart tambahan, sehingga terdapat 4 pendakian.; max_restart 3; iterasi per run [2, 2, 1, 1]

Percobaan 2: seed 2028 menaikkan objective 205 menjadi 235 Kendaraan akhir yang dimuat: B, C, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

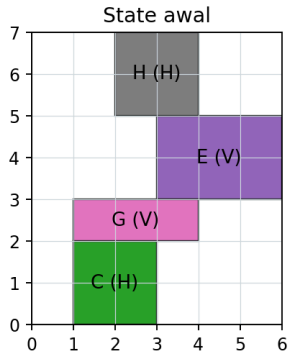
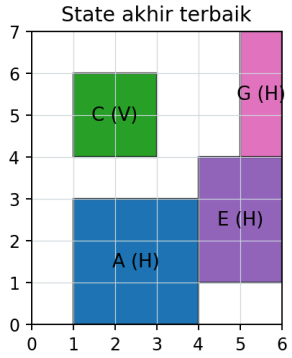
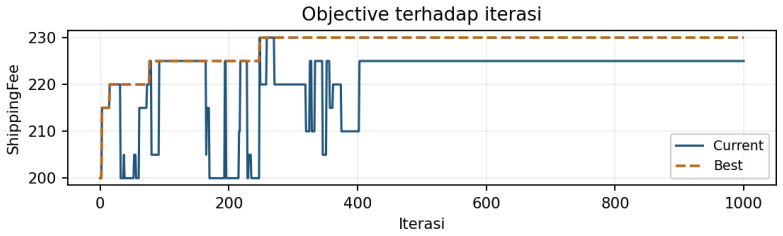
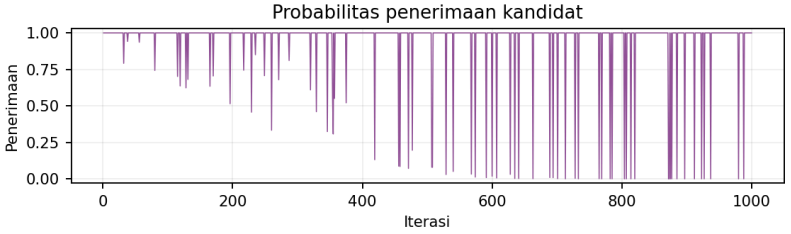
Tabel 31. Hasil Random Restart Hill Climbing , Percobaan 3

Percobaan 3	
State Awal	 **Muatan dan nilai** Inside: C, E, F Outside: A, B, D, G, H ShippingFee: 210 Berat: 18 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H = horizontal; V = vertical (0,0) di kiri bawah
Nilai Objective	235 (awal 210); valid: ya
Plot Objective per Iterasi	
Durasi Pencarian	0.0298 detik (diukur menggunakan time.perf_counter)
Iterasi dan Parameter	4 iterasi; seed 2029; Batas 100 iterasi per pendakian; maksimum 3 restart tambahan, sehingga terdapat 4 pendakian.; max_restart 3; iterasi per run [1, 1, 1, 1]

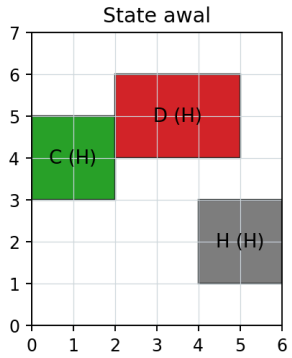
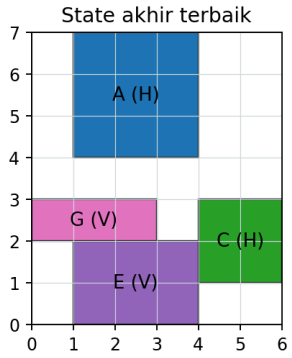
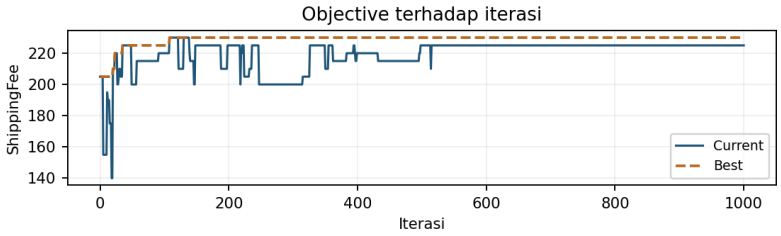
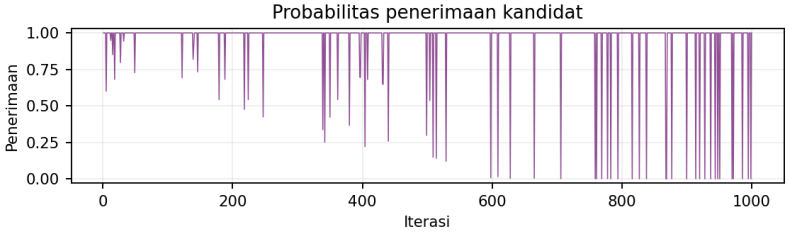
Percobaan 3: seed 2029 menaikkan objective 210 menjadi 235. Kendaraan akhir yang dimuat: B, C, F. Seluruh kendaraan terwakili dan state awal serta akhir lolos pemeriksaan batas, tabrakan, dan kapasitas.

2.4.6 Hasil Simulated Annealing

Tabel 32. Hasil Simulated Annealing , Percobaan 1

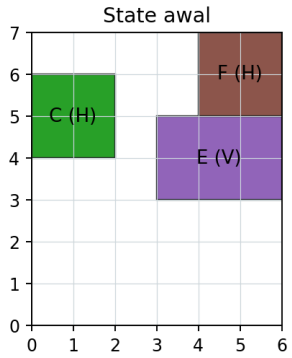
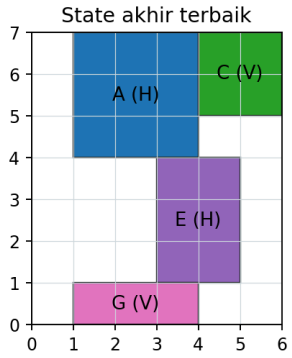
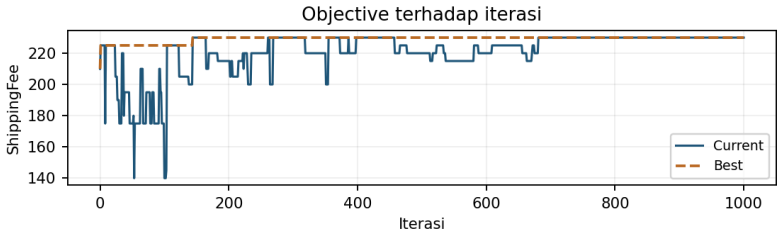
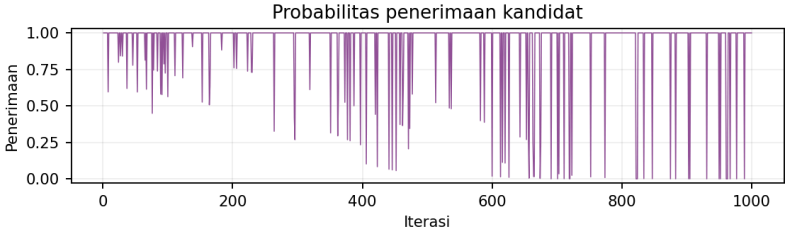
Percobaan 1; seed 2027	
State Awal	 **Muatan dan nilai** Inside: C, E, G, H Outside: A, B, D, F ShippingFee: 200 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: A, C, E, G Outside: B, D, F, H ShippingFee: 230 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	230 (awal 200); kedua state valid
Plot Objective per Iterasi	
Plot Probabilitas Penerimaan	
Frekuensi Stuck	38 kali (indikator: Frekuensi stuck dioperasionalkan sebagai jumlah kejadian ketika pencarian melewati 25 iterasi berturut-turut tanpa peningkatan nilai best-so-far. Ukuran ini menunjukkan stagnasi pencarian dan tidak memastikan bahwa state berada pada optimum lokal.)
Durasi dan Iterasi	0.0577 detik; 1000 iterasi; accepted_moves 947

Tabel 33. Hasil Simulated Annealing , Percobaan 2

Percobaan 2; seed 2028	
State Awal	 **Muatan dan nilai** Inside: C, D, H Outside: A, B, E, F, G ShippingFee: 205 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: A, C, E, G Outside: B, D, F, H ShippingFee: 230 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	230 (awal 205); kedua state valid
Plot Objective per Iterasi	
Plot Probabilitas Penerimaan	
Frekuensi Stuck	37 kali (indikator: Frekuensi stuck dioperasionalkan sebagai jumlah kejadian ketika pencarian melewati 25 iterasi berturut-turut tanpa peningkatan nilai best-so-far. Ukuran ini menunjukkan stagnasi pencarian dan tidak memastikan bahwa state berada pada optimum lokal.)
Durasi dan Iterasi	0.0535 detik; 1000 iterasi; accepted_moves 958

Percobaan 2 (seed 2028): objective 205 menjadi 230. Muatan akhir A, C, E, G; stuck_count 37. State awal dan akhir valid; gambar ditata ulang dari history dan state run yang sama.

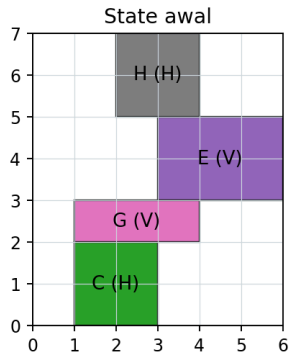
Tabel 34. Hasil Simulated Annealing , Percobaan 3

Percobaan 3; seed 2029	
State Awal	 **Muatan dan nilai** Inside: C, E, F Outside: A, B, D, G, H ShippingFee: 210 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: A, C, E, G Outside: B, D, F, H ShippingFee: 230 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	230 (awal 210); kedua state valid
Plot Objective per Iterasi	
Plot Probabilitas Penerimaan	
Frekuensi Stuck	39 kali (indikator: Frekuensi stuck dioperasionalkan sebagai jumlah kejadian ketika pencarian melewati 25 iterasi berturut-turut tanpa peningkatan nilai best-so-far. Ukuran ini menunjukkan stagnasi pencarian dan tidak memastikan bahwa state berada pada optimum lokal.)
Durasi dan Iterasi	0.0486 detik; 1000 iterasi; accepted_moves 937

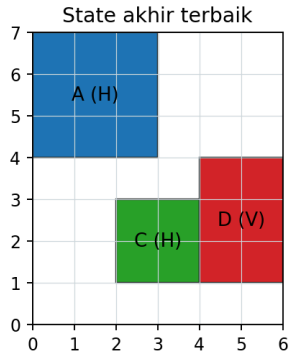
Percobaan 3 (seed 2029): objective 210 menjadi 230. Muatan akhir A, C, E, G; stuck_count 39. Ketiga run SA mencapai 230. Stuck_count ialah indikator stagnasi menurut program, bukan bukti optimum lokal. Kurva current dapat turun walaupun best tidak menurun.

2.4.7 Hasil Genetic Algorithm dengan Variasi Generasi

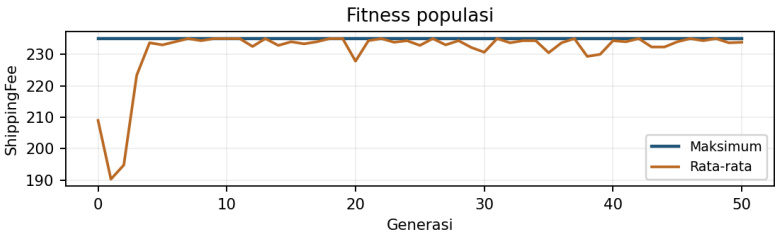
Tabel 35. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 1, Percobaan 1 (50 generasi; 30 populasi)

<table>
<tr><td colspan="2">Konfigurasi 1, percobaan 1; seed 2027</td></tr>
<tr><td>State Awal</td><td> 

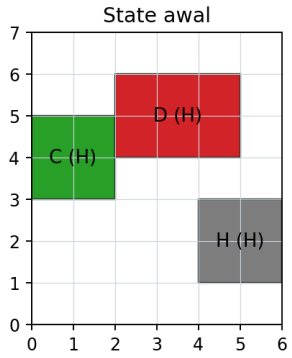
Muatan dan nilai

Inside: C, E, G, H
Outside: A, B, D, F
ShippingFee: 200
Berat: 18 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>State Akhir</td><td> 

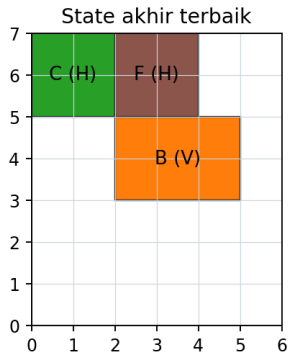
Muatan dan nilai

Inside: A, C, D
Outside: B, E, F, G, H
ShippingFee: 235
Berat: 20 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>Nilai Objective</td><td>235 (state awal 200; maksimum populasi awal 235); valid</td></tr>
<tr><td>Plot Maksimum dan Rata Rata</td><td> 
</td></tr>
<tr><td>Jumlah Populasi dan Generasi</td><td>30 individu; 50 generasi; generasi 0 juga digambar</td></tr>
<tr><td>Durasi Pencarian</td><td>0.0762 detik (loop generasi saja)</td></tr>
</table>

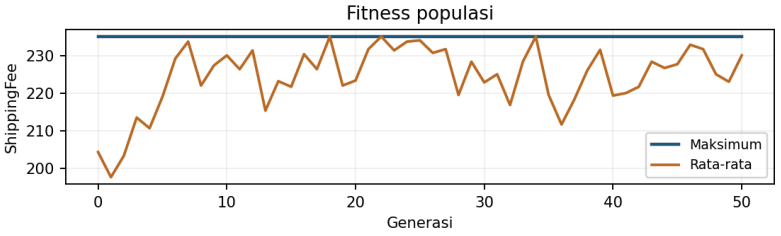
Tabel 36. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 1, Percobaan 2 (50 generasi; 30 populasi)

<table>
<tr><td colspan="2">Konfigurasi 1, percobaan 2; seed 2028</td></tr>
<tr><td>State Awal</td><td> 

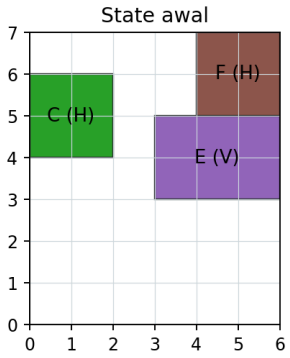
Muatan dan nilai

Inside: C, D, H
Outside: A, B, E, F, G
ShippingFee: 205
Berat: 18 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>State Akhir</td><td> 

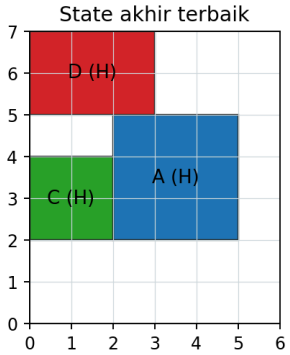
Muatan dan nilai

Inside: B, C, F
Outside: A, D, E, G, H
ShippingFee: 235
Berat: 20 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>Nilai Objective</td><td>235 (state awal 205; maksimum populasi awal 235); valid</td></tr>
<tr><td>Plot Maksimum dan Rata Rata</td><td> 
</td></tr>
<tr><td>Jumlah Populasi dan Generasi</td><td>30 individu; 50 generasi; generasi 0 juga digambar</td></tr>
<tr><td>Durasi Pencarian</td><td>0.0729 detik (loop generasi saja)</td></tr>
</table>

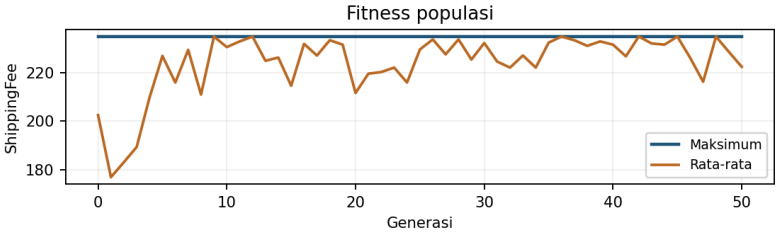
Tabel 37. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 1, Percobaan 3 (50 generasi; 30 populasi)

<table>
<tr><td colspan="2">Konfigurasi 1, percobaan 3; seed 2029</td></tr>
<tr><td>State Awal</td><td> 

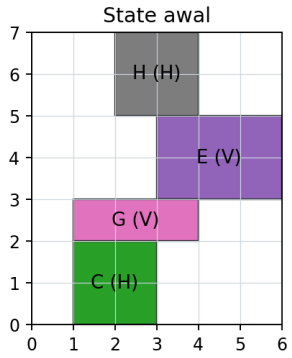
Muatan dan nilai

Inside: C, E, F
Outside: A, B, D, G, H
ShippingFee: 210
Berat: 18 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>State Akhir</td><td> 

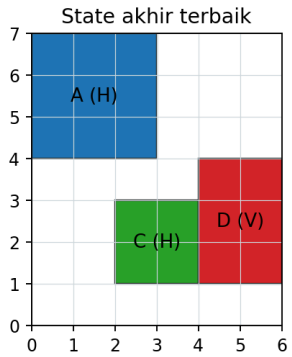
Muatan dan nilai

Inside: A, C, D
Outside: B, E, F, G, H
ShippingFee: 235
Berat: 20 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>Nilai Objective</td><td>235 (state awal 210; maksimum populasi awal 235); valid</td></tr>
<tr><td>Plot Maksimum dan Rata Rata</td><td> 
</td></tr>
<tr><td>Jumlah Populasi dan Generasi</td><td>30 individu; 50 generasi; generasi 0 juga digambar</td></tr>
<tr><td>Durasi Pencarian</td><td>0.0706 detik (loop generasi saja)</td></tr>
</table>

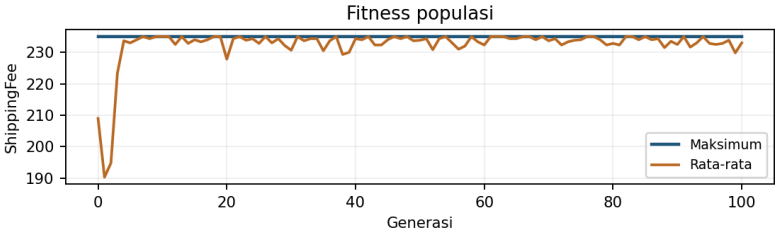
Tabel 38. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 2, Percobaan 1 (100 generasi; 30 populasi)

<table>
<tr><td colspan="2">Konfigurasi 2, percobaan 1; seed 2027</td></tr>
<tr><td>State Awal</td><td> 

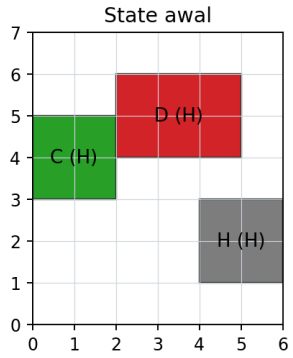
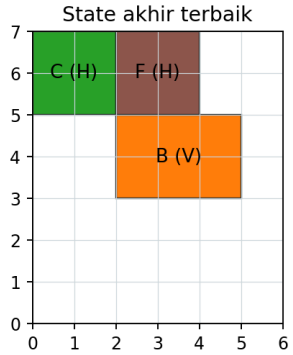
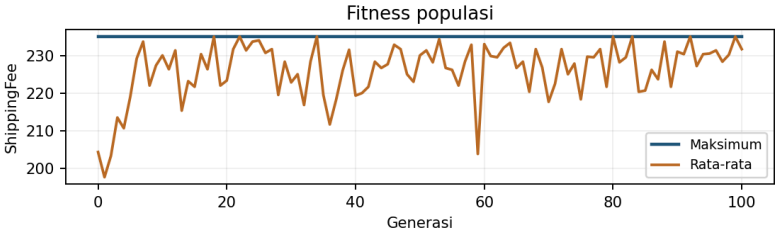
Muatan dan nilai

Inside: C, E, G, H
Outside: A, B, D, F
ShippingFee: 200
Berat: 18 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>State Akhir</td><td> 

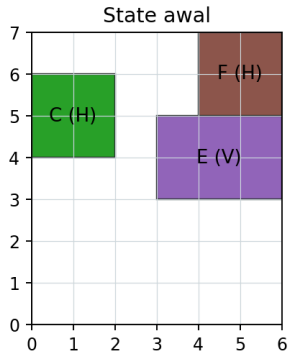
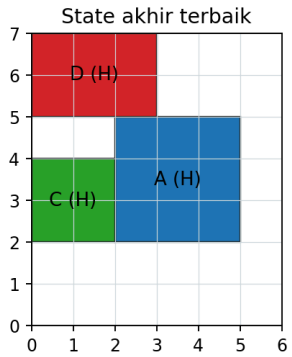
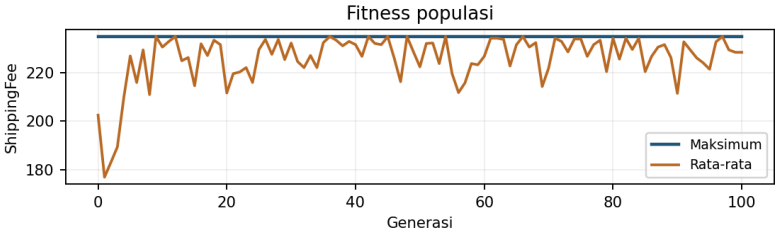
Muatan dan nilai

Inside: A, C, D
Outside: B, E, F, G, H
ShippingFee: 235
Berat: 20 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>Nilai Objective</td><td>235 (state awal 200; maksimum populasi awal 235); valid</td></tr>
<tr><td>Plot Maksimum dan Rata Rata</td><td> 
</td></tr>
<tr><td>Jumlah Populasi dan Generasi</td><td>30 individu; 100 generasi; generasi 0 juga digambar</td></tr>
<tr><td>Durasi Pencarian</td><td>0.1598 detik (loop generasi saja)</td></tr>
</table>

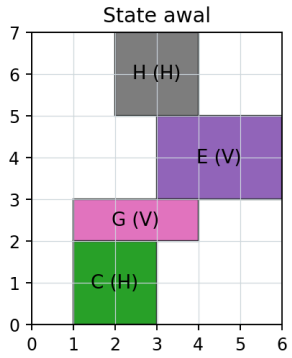
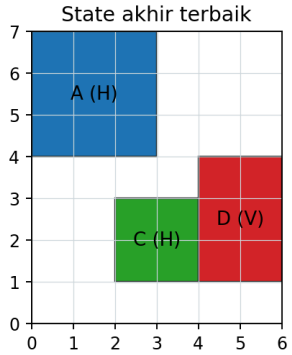
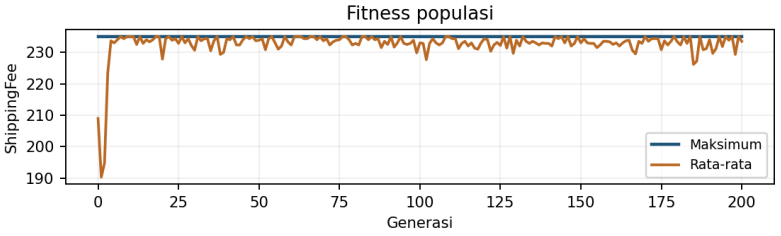
Tabel 39. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 2, Percobaan 2 (100 generasi; 30 populasi)

Konfigurasi 2, percobaan 2; seed 2028	
State Awal	 **Muatan dan nilai** Inside: C, D, H Outside: A, B, E, F, G ShippingFee: 205 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 205; maksimum populasi awal 235); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	30 individu; 100 generasi; generasi 0 juga digambar
Durasi Pencarian	0.1317 detik (loop generasi saja)

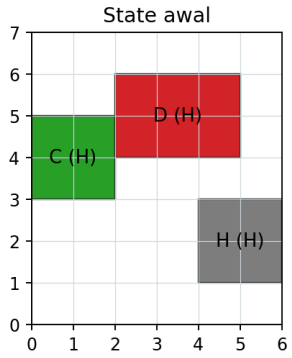
Tabel 40. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 2, Percobaan 3 (100 generasi; 30 populasi)

Konfigurasi 2, percobaan 3; seed 2029	
State Awal	 **Muatan dan nilai** Inside: C, E, F Outside: A, B, D, G, H ShippingFee: 210 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: A, C, D Outside: B, E, F, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 210; maksimum populasi awal 235); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	30 individu; 100 generasi; generasi 0 juga digambar
Durasi Pencarian	0.2111 detik (loop generasi saja)

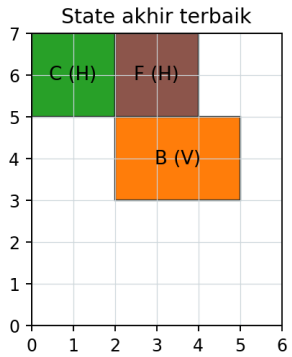
Tabel 41. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 3, Percobaan 1 (200 generasi; 30 populasi)

Konfigurasi 3, percobaan 1; seed 2027	
State Awal	 **Muatan dan nilai** Inside: C, E, G, H Outside: A, B, D, F ShippingFee: 200 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: A, C, D Outside: B, E, F, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 200; maksimum populasi awal 235); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	30 individu; 200 generasi; generasi 0 juga digambar
Durasi Pencarian	0.3418 detik (loop generasi saja)

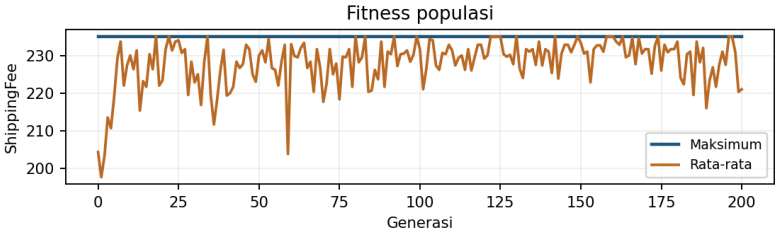
Tabel 42. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 3, Percobaan 2 (200 generasi; 30 populasi)

<table>
<tr><td colspan="2">Konfigurasi 3, percobaan 2; seed 2028</td></tr>
<tr><td>State Awal</td><td> 

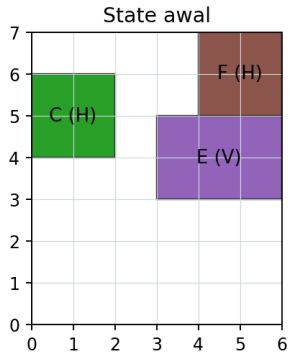
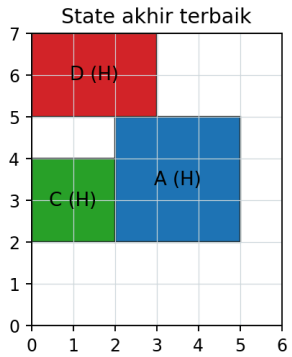
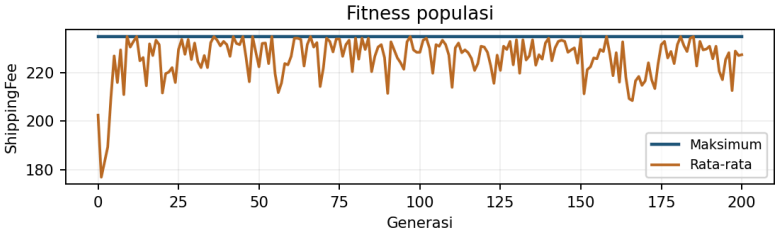
Muatan dan nilai

Inside: C, D, H
Outside: A, B, E, F, G
ShippingFee: 205
Berat: 18 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>State Akhir</td><td> 

Muatan dan nilai

Inside: B, C, F
Outside: A, D, E, G, H
ShippingFee: 235
Berat: 20 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>Nilai Objective</td><td>235 (state awal 205; maksimum populasi awal 235); valid</td></tr>
<tr><td>Plot Maksimum dan Rata Rata</td><td> 
</td></tr>
<tr><td>Jumlah Populasi dan Generasi</td><td>30 individu; 200 generasi; generasi 0 juga digambar</td></tr>
<tr><td>Durasi Pencarian</td><td>0.2843 detik (loop generasi saja)</td></tr>
</table>

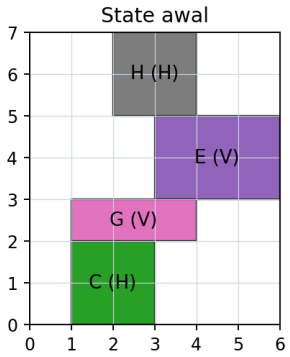
Tabel 43. Hasil Genetic Algorithm dengan Variasi Generasi , Konfigurasi 3, Percobaan 3 (200 generasi; 30 populasi)

Konfigurasi 3, percobaan 3; seed 2029	
State Awal	 **Muatan dan nilai** Inside: C, E, F Outside: A, B, D, G, H ShippingFee: 210 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: A, C, D Outside: B, E, F, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 210; maksimum populasi awal 235); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	30 individu; 200 generasi; generasi 0 juga digambar
Durasi Pencarian	0.3006 detik (loop generasi saja)

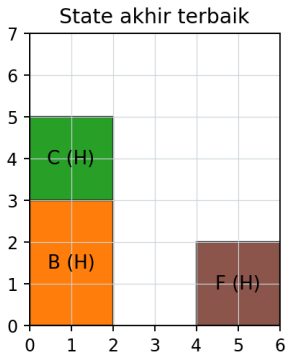
Ketika ukuran populasi dijaga 30, ketiga variasi (50, 100, 200 generasi) masing-masing diuji tiga kali. Semua hasil berakhir pada 235, nilai itu sudah ada pada populasi generasi 0 di seluruh sembilan run, sehingga data ini tidak menunjukkan kenaikan best akibat tambahan generasi. Grafik tetap memperlihatkan perubahan rata-rata fitness sepanjang proses. Analisis perbandingan hasil eksperimen disajikan pada Subbab 2.5.

2.4.8 Hasil Genetic Algorithm dengan Variasi Populasi

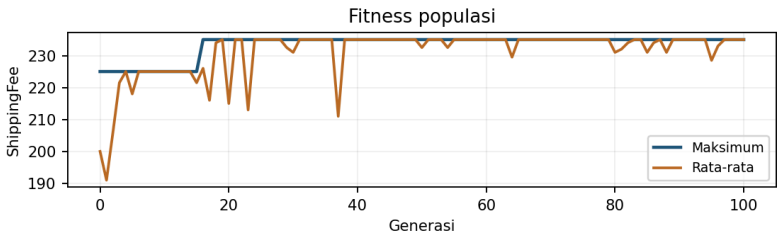
Tabel 44. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 1, Percobaan 1 (10 populasi; 100 generasi)

<table>
<tr><td colspan="2">Konfigurasi 1, percobaan 1; seed 2027</td></tr>
<tr><td>State Awal</td><td> 

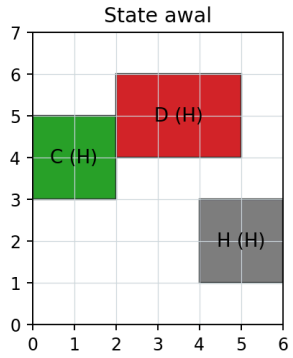
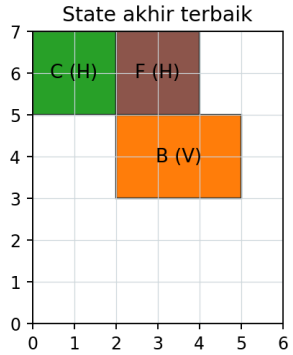
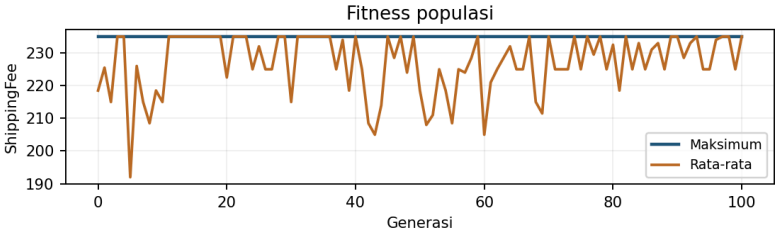
Muatan dan nilai

Inside: C, E, G, H
Outside: A, B, D, F
ShippingFee: 200
Berat: 18 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>State Akhir</td><td> 

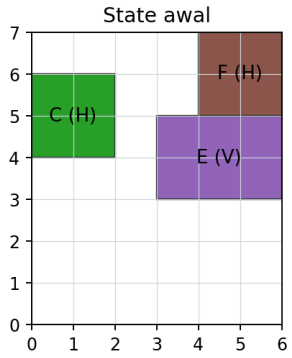
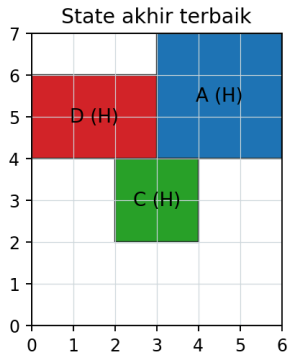
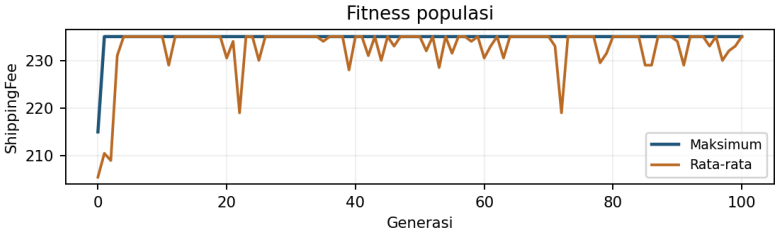
Muatan dan nilai

Inside: B, C, F
Outside: A, D, E, G, H
ShippingFee: 235
Berat: 20 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>Nilai Objective</td><td>235 (state awal 200; maksimum populasi awal 225); valid</td></tr>
<tr><td>Plot Maksimum dan Rata Rata</td><td> 
</td></tr>
<tr><td>Jumlah Populasi dan Generasi</td><td>10 individu; 100 generasi; generasi 0 juga digambar</td></tr>
<tr><td>Durasi Pencarian</td><td>0.0400 detik (loop generasi saja)</td></tr>
</table>

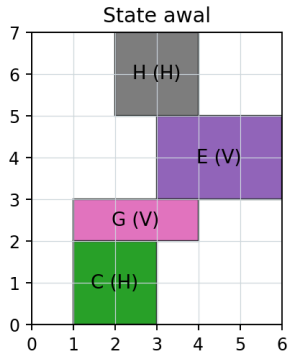
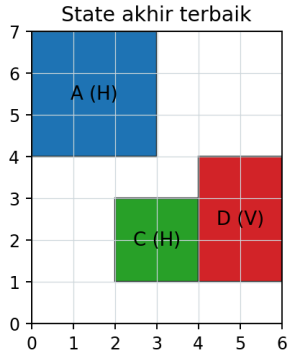
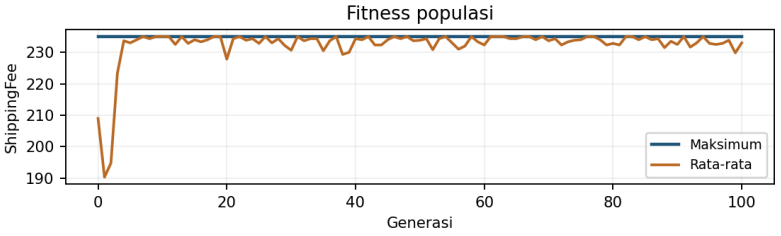
Tabel 45. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 1, Percobaan 2 (10 populasi; 100 generasi)

Konfigurasi 1, percobaan 2; seed 2028	
State Awal	 **Muatan dan nilai** Inside: C, D, H Outside: A, B, E, F, G ShippingFee: 205 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 205; maksimum populasi awal 235); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	10 individu; 100 generasi; generasi 0 juga digambar
Durasi Pencarian	0.0420 detik (loop generasi saja)

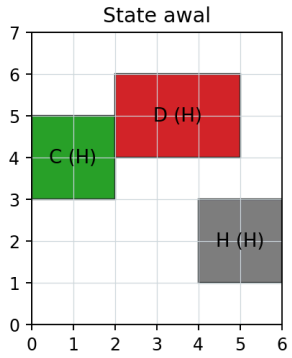
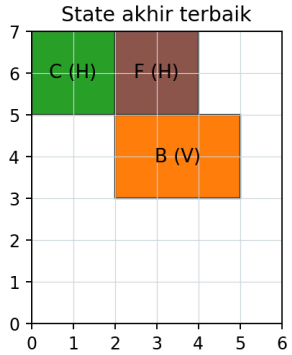
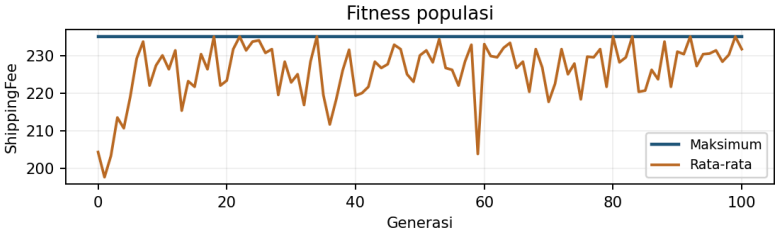
Tabel 46. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 1, Percobaan 3 (10 populasi; 100 generasi)

Konfigurasi 1, percobaan 3; seed 2029	
State Awal	 **Muatan dan nilai** Inside: C, E, F Outside: A, B, D, G, H ShippingFee: 210 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: A, C, D Outside: B, E, F, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 210; maksimum populasi awal 215); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	10 individu; 100 generasi; generasi 0 juga digambar
Durasi Pencarian	0.0452 detik (loop generasi saja)

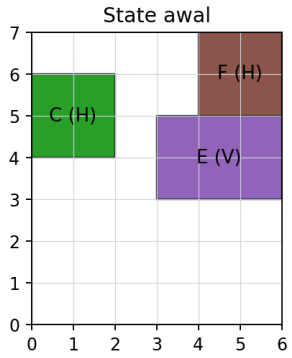
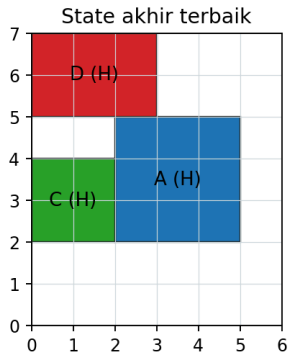
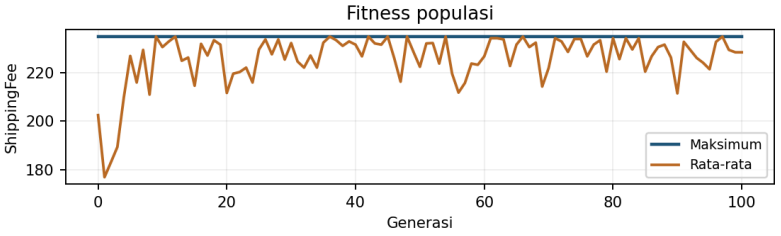
Tabel 47. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 2, Percobaan 1 (30 populasi; 100 generasi)

Konfigurasi 2, percobaan 1; seed 2027	
State Awal	 **Muatan dan nilai** Inside: C, E, G, H Outside: A, B, D, F ShippingFee: 200 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: A, C, D Outside: B, E, F, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 200; maksimum populasi awal 235); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	30 individu; 100 generasi; generasi 0 juga digambar
Durasi Pencarian	0.1407 detik (loop generasi saja)

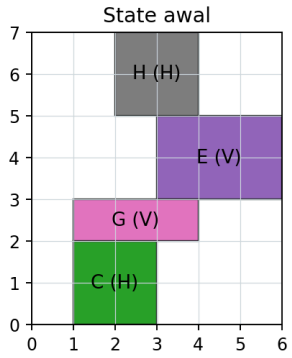
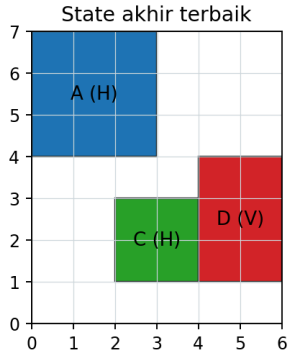
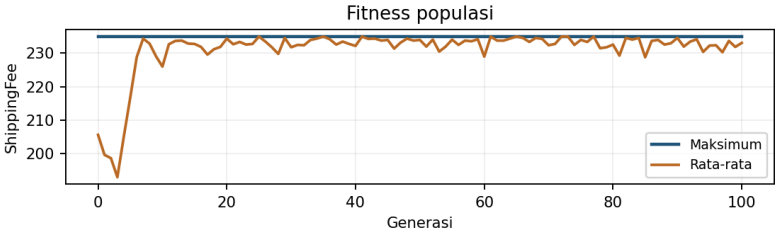
Tabel 48. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 2, Percobaan 2 (30 populasi; 100 generasi)

Konfigurasi 2, percobaan 2; seed 2028	
State Awal	 **Muatan dan nilai** Inside: C, D, H Outside: A, B, E, F, G ShippingFee: 205 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 205; maksimum populasi awal 235); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	30 individu; 100 generasi; generasi 0 juga digambar
Durasi Pencarian	0.1426 detik (loop generasi saja)

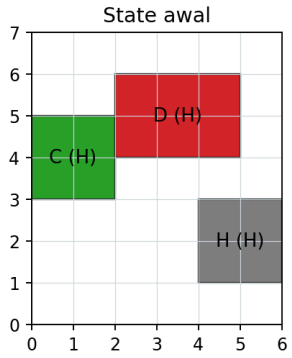
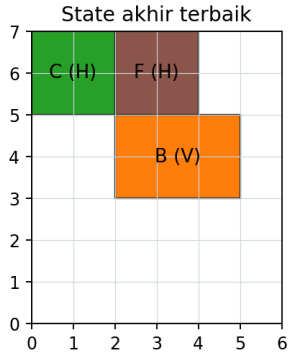
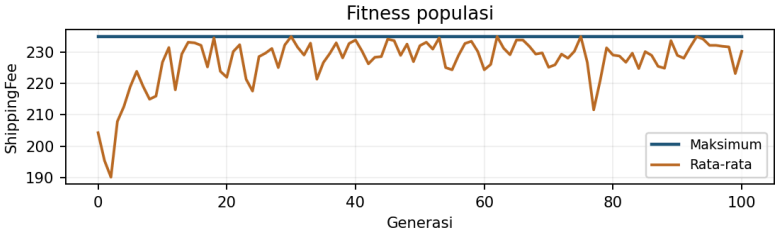
Tabel 49. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 2, Percobaan 3 (30 populasi; 100 generasi)

Konfigurasi 2, percobaan 3; seed 2029	
State Awal	 **Muatan dan nilai** Inside: C, E, F Outside: A, B, D, G, H ShippingFee: 210 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: A, C, D Outside: B, E, F, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 210; maksimum populasi awal 235); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	30 individu; 100 generasi; generasi 0 juga digambar
Durasi Pencarian	0.1429 detik (loop generasi saja)

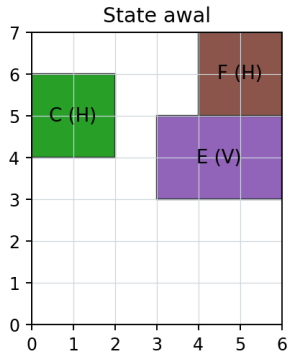
Tabel 50. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 3, Percobaan 1 (50 populasi; 100 generasi)

Konfigurasi 3, percobaan 1; seed 2027	
State Awal	 **Muatan dan nilai** Inside: C, E, G, H Outside: A, B, D, F ShippingFee: 200 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: A, C, D Outside: B, E, F, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 200; maksimum populasi awal 235); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	50 individu; 100 generasi; generasi 0 juga digambar
Durasi Pencarian	0.2531 detik (loop generasi saja)

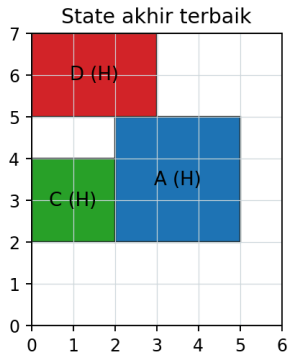
Tabel 51. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 3, Percobaan 2 (50 populasi; 100 generasi)

Konfigurasi 3, percobaan 2; seed 2028	
State Awal	 **Muatan dan nilai** Inside: C, D, H Outside: A, B, E, F, G ShippingFee: 205 Berat: 18 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
State Akhir	 **Muatan dan nilai** Inside: B, C, F Outside: A, D, E, G, H ShippingFee: 235 Berat: 20 / 20 Valid: Ya H/V: orientasi (0,0): kiri bawah
Nilai Objective	235 (state awal 205; maksimum populasi awal 235); valid
Plot Maksimum dan Rata Rata	
Jumlah Populasi dan Generasi	50 individu; 100 generasi; generasi 0 juga digambar
Durasi Pencarian	0.2594 detik (loop generasi saja)

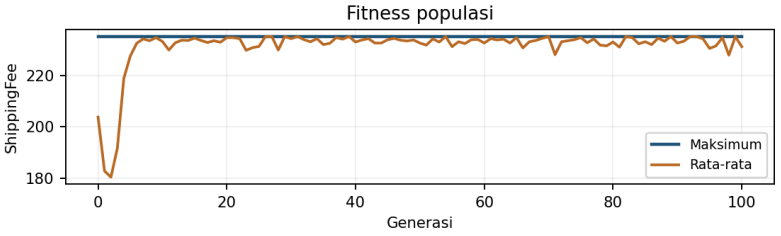
Tabel 52. Hasil Genetic Algorithm dengan Variasi Populasi , Konfigurasi 3, Percobaan 3 (50 populasi; 100 generasi)

<table>
<tr><td colspan="2">Konfigurasi 3, percobaan 3; seed 2029</td></tr>
<tr><td>State Awal</td><td> 

Muatan dan nilai

Inside: C, E, F
Outside: A, B, D, G, H
ShippingFee: 210
Berat: 18 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>State Akhir</td><td> 

Muatan dan nilai

Inside: A, C, D
Outside: B, E, F, G, H
ShippingFee: 235
Berat: 20 / 20
Valid: Ya
H/V: orientasi
(0,0): kiri bawah</td></tr>
<tr><td>Nilai Objective</td><td>235 (state awal 210; maksimum populasi awal 235); valid</td></tr>
<tr><td>Plot Maksimum dan Rata Rata</td><td> 
</td></tr>
<tr><td>Jumlah Populasi dan Generasi</td><td>50 individu; 100 generasi; generasi 0 juga digambar</td></tr>
<tr><td>Durasi Pencarian</td><td>0.2575 detik (loop generasi saja)</td></tr>
</table>

Ketika jumlah generasi dijaga 100, ukuran populasi 10, 30, dan 50 masing-masing diuji tiga kali. Semua hasil berakhir pada 235. Untuk populasi 10 pada seed 2027 dan 2029, maksimum populasi awal hanya 225 dan 215, lalu meningkat menjadi 235, pada populasi 30 dan 50, 235 sudah ada sejak generasi 0. Tiga percobaan per konfigurasi ini tidak cukup untuk menyimpulkan kinerja umum semua ukuran populasi.

2.4.9 Hasil Fungsi Objektif Alternatif

Tabel 53. Hasil Fungsi Objektif Alternatif

Fungsi Objektif	Algoritma	Percobaan	Hasil
shippingFee / (width × length)	Steepest HC	3 run; seed 2027–2029	Skor: 52.917; 56.250; 56.250; fee mentah: 225; 225; 225. Valid 3/3.
shippingFee / (width × length)	GA p30 g100	3 run; seed 2027–2029	Skor: 56.250; 56.250; 56.250; fee mentah: 225; 225; 225. Valid 3/3.
shippingFee / (1 + ETA)	Steepest HC	3 run; seed 2027–2029	Skor: 86.250; 151.250; 151.250; fee mentah: 225; 235; 235. Valid 3/3.
shippingFee / (1 + ETA)	GA p30 g100	3 run; seed 2027–2029	Skor: 151.250; 151.250; 151.250; fee mentah: 235; 235; 235. Valid 3/3.

Pada input satu kapal yang sama, mode fee_per_area memberi bobot tinggi untuk fee per luas kendaraan, sedangkan fee_with_eta memberi bobot tinggi untuk fee dengan ETA lebih kecil. Skor dalam kedua mode mempunyai skala berbeda dari shipping_fee; kolom fee mentah ditampilkan sebagai pembanding rupiah satuan input tanpa menyamakan nilainya dengan skor alternatif. HC steepest dibatasi 100 iterasi; GA memakai populasi 30, 100 generasi, crossover 0,8, mutasi 0,2, elitisme 2 dan tournament 3. Masing-masing dijalankan pada seed 2027–2029. Semua 12 state akhir valid dan memuat delapan Placement. Data setiap percobaan, state hasil pencarian, dan gambar keluaran disertakan dalam berkas pendukung yang dirinci pada Lampiran B.

2.4.10 Hasil Penempatan Multi-Kapal

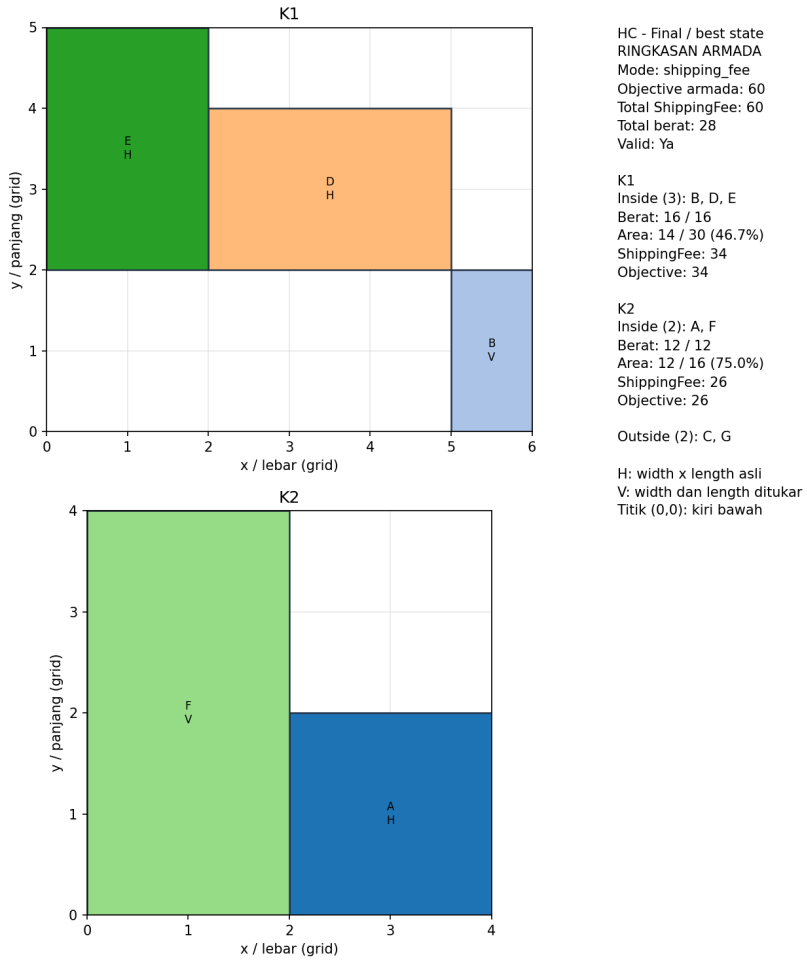
Tabel 54. Hasil Penempatan Multi-Kapal

Algoritma	State Awal	State Akhir	Nilai Objective
Steepest HC	42 / 52 / 34	60 / 52 / 56	Seed 2027/28/29; valid 3/3. Best 60.
SA	42 / 52 / 34	60 / 58 / 60	Seed 2027/28/29; valid 3/3. Best 60.
GA p30 g100	42 / 52 / 34	60 / 60 / 58	Seed 2027/28/29; valid 3/3. Best 60.
K1; contoh HC 2027	K1, kapasitas 16	B, D, E; berat 16	Fee K1 = 34; valid
K2; contoh HC 2027	K2, kapasitas 12	A, F; berat 12	Fee K2 = 26; valid

Input dua kapal pada repository memuat tujuh kendaraan A–G. Tiap algoritma diuji pada seed 2027–2029 dengan mode shipping_fee; HC steepest maksimum 100 iterasi, SA 1000 iterasi dengan T0 100 dan pendinginan 0,995, GA populasi 30 dan 100 generasi. Pada HC seed 2027, K1 memuat B, D, E (16/16) dan K2 memuat A, F (12/12), sehingga total shipping fee 60; C dan G outside. Semua sembilan keluaran valid, mencatat ketujuh kendaraan, serta memenuhi kapasitas tiap kapal. Angka 60 ialah optimum untuk input armada ini: G tidak muat akibat dimensi, dan enumerasi penugasan A–F dengan batas kapasitas saja memberi batas atas 60; solusi valid mencapai batas tersebut. Pembuktian enumerasi dicatat di Lampiran B.

Gambar 2. Visualisasi Penempatan Multi-Kapal

HC Steepest, seed 2027: state awal dan state akhir (dua kapal)



Gambar 2 memperlihatkan state awal dan akhir HC steepest, input/multi_ship_example.json, seed 2027. Gambar asli dihasilkan oleh src/visualization/grid.py melalui export_result; di kanan tiap gambar terlihat rekap per kapal, kendaraan outside, objective, dan status valid.

2.5 Analisis Hasil Eksperimen

2.5.1 Kualitas Solusi dan Kedekatan dengan Optimum

Tabel 55. Analisis Kualitas Solusi dan Kedekatan dengan Optimum

Aspek	Hasil Pengamatan	Penjelasan
Batas satu kapal	235	Enumerasi seluruh subset A–H dengan total berat ≤ 20, tanpa syarat geometri, memberi batas atas 235.
Solusi layak	235 dari A,C,D atau B,C,F	State keluaran valid dengan fee 235 mencapai batas atas, maka optimum global khusus input ini 235.
Pencapaian	HC: 1/3 hingga 3/3; SA: 0/3; GA: 18/18	Hitung run bernilai 235 pada masing-masing konfigurasi; bukan jaminan untuk input lain.
Selisih SA	230/235 = 97,87%	SA tertinggal 5 fee atau sekitar 2,13% dari optimum pada tiga seed yang diuji.

Bukti optimum 235 berlaku hanya pada dataset satu kapal yang digunakan dalam percobaan. Setiap solusi valid merupakan subset kendaraan berbobot paling banyak 20; enumerasi semua 256 subset menemukan maksimum shipping fee 235 pada A,C,D dan B,C,F. Karena setidaknya satu penempatan aktual untuk subset tersebut lolos is_valid_state, batas atas itu tercapai. Ini membuktikan optimum untuk input dan objective shipping_fee tersebut, bukan untuk persoalan umum.

2.5.2 Perbandingan Lima Varian Hill Climbing

Tabel 56. Analisis Perbandingan Lima Varian Hill Climbing

Aspek	Hasil Pengamatan	Penjelasan
First Improvement	225 / 235 / 225; rerata 228,33	Pertama yang membaik; optimum 1 dari 3 run.
Steepest dan Stochastic	Masing-masing 225 / 235 / 235; rerata 231,67	Steepest memilih maksimum tetangga; stochastic memilih acak tetangga yang membaik.
Sideways Move	225 / 235 / 235; rerata 231,67	Empat sideways move pada tiap run; pada data ini skor akhirnya sama dengan steepest.
Random Restart	235 / 235 / 235; rerata 235	Tiga restart tambahan, memilih hasil terbaik dari empat pencarian; optimum 3/3.

Semua varian berangkat dari state awal yang sama untuk seed yang sama (200, 205, 210) dan dibatasi 100 iterasi per pencarian. First Improvement mencapai optimum sekali; Steepest, Stochastic, dan Sideways Move dua kali; Random Restart tiga kali. Pada sampel ini sideways yang diperbolehkan empat kali tidak menaikkan skor dibanding Steepest, sedangkan restart memperluas titik awal pencarian. Kesimpulan ini terbatas pada tiga seed.

2.5.3 Perbandingan Hill Climbing, Simulated Annealing, dan Genetic Algorithm

Tabel 57. Analisis Perbandingan Hill Climbing, Simulated Annealing, dan Genetic Algorithm

Aspek	Hasil Pengamatan	Penjelasan
HC terbaik	Random Restart: 235 / 235 / 235	Mencapai optimum 3/3; ini konfigurasi HC yang memakai pencarian berulang.
SA	230 / 230 / 230	Tiga run pada T0 100, pendinginan 0,995, 1000 iterasi; optimum 0/3.
GA	235 pada seluruh 18 run	Untuk semua variasi p dan g yang diuji; sebagian optimum sudah ada di populasi awal.
Kesetaraan data	Input dan seed 2027–2029 sama	SA dan GA menginisialisasi state tunggal sama; GA membuat populasi tambahan dari RNG internal.

Pada dataset satu kapal, HC Random Restart dan seluruh konfigurasi GA mencapai nilai optimum 235 pada tiga seed; SA mencapai 230. Perbandingan angka akhir menggunakan objective shipping_fee dan batas yang sama. Mekanisme dan anggaran pencarian berbeda: GA mengevaluasi populasi, SA menjalankan 1000 iterasi, dan Random Restart mengulangi pendakian. Keunggulan hasil dalam sampel kecil ini tidak membuktikan keunggulan umum salah satu algoritma.

2.5.4 Perbandingan Durasi dan Konsistensi Hasil

Tabel 58. Analisis Perbandingan Durasi dan Konsistensi Hasil

Aspek	Hasil Pengamatan	Penjelasan
HC, rerata detik	Steepest 0,0079; First 0,0106; Stochastic 0,0122	Setiap varian Hill Climbing diuji sebanyak tiga kali. Nilai yang ditampilkan merupakan rata-rata dari ketiga percobaan.
HC berulang	Sideways 0,0203; Restart 0,0297	Lebih banyak evaluasi tetangga atau restart pada data ini.

Aspek	Hasil Pengamatan	Penjelasan
SA dan GA	SA 0,0533; GA p30 g100 sekitar 0,16	Simulated Annealing diuji sebanyak tiga kali. Genetic Algorithm diuji dalam dua seri dengan ukuran populasi 30 dan jumlah generasi 100.
Batas durasi	Timer GA mulai setelah populasi awal	Waktu GA tidak mencakup pembentukan populasi awal sehingga belum merepresentasikan keseluruhan waktu eksekusi.

Nilai akhir Random Restart dan GA konsisten 235 pada tiga seed, sementara SA konsisten 230; konsistensi yang sama bisa terjadi pada kualitas berbeda. Perbandingan durasi hanya deskriptif karena angka kecil dipengaruhi lingkungan pengukuran dan fungsi GA memulai timer setelah populasi awal dibentuk (src/algorithms/genetic_algorithm.py baris 114–129). Perbandingan waktu eksekusi secara menyeluruh memerlukan pengukuran sejak sebelum inisialisasi hingga algoritma selesai, menggunakan mesin dan prosedur pengujian yang sama.

2.5.5 Pengaruh Generasi dan Populasi pada Genetic Algorithm

Tabel 59. Analisis Pengaruh Generasi dan Populasi pada Genetic Algorithm

Aspek	Hasil Pengamatan	Penjelasan
Generasi 50/100/200	p=30; semua best 235	Pada seluruh sembilan run, populasi generasi 0 sudah mengandung 235.
Durasi generasi	Rerata 0,0732 / 0,1675 / 0,3089 detik	Sesuai kenaikan pekerjaan dalam loop; belum termasuk populasi awal.
Populasi 10/30/50	g=100; semua best 235	Pada p10, seed 2027 dan 2029 naik dari max awal 225 dan 215 ke 235.
Durasi populasi	Rerata 0,0424 / 0,1420 / 0,2567 detik	p30 dan p50 sudah memiliki best 235 pada generasi 0 dalam semua run.

Penambahan generasi dari 50 ke 200 pada populasi 30 tidak menaikkan best akhir karena setiap populasi awal dalam seri itu telah memuat optimum. Perubahan populasi 10 ke 30/50 juga tidak mengubah best akhir; pada populasi 10, dua dari tiga run justru memperlihatkan perbaikan dari generasi awal. Grafik rata-rata fitness dapat berubah walau best tetap. Hasil ini hanya mencakup nilai parameter dan seed yang ditampilkan.

2.5.6 Analisis Fungsi Objektif Alternatif dan Multi-Kapal

Tabel 60. Analisis Fungsi Objektif Alternatif dan Multi-Kapal

Aspek	Hasil Pengamatan	Penjelasan
Fee per luas	HC 52,917/56,25/56,25; GA 56,25 × 3	Fee mentah seluruh solusi akhir 225; skor alternatif tidak seunit dengan fee.
Fee dan ETA	HC 86,25/151,25/151,25; GA 151,25 × 3	Fee mentah HC 225/235/235; GA 235 × 3.
Armada dua kapal	HC 60/52/56; SA 60/58/60; GA 60/60/58	Semua sah; nilai 60 mencapai batas atas khusus input dua kapal.
Cakupan	12 run alternatif; 9 run armada	Mode alternatif memakai input satu kapal, armada memakai input repo; skor tidak dibandingkan langsung lintas input.

Dua mode alternatif mengubah preferensi solusi sesuai rumus di src/core/objective.py; skornya tidak dapat dibandingkan langsung antar-mode. Fee mentah ditampilkan terpisah. Pada input dua kapal, keluaran tetap valid, walau SA dan GA tidak selalu mencapai 60 pada setiap seed.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan keseluruhan hasil eksperimen, dapat disimpulkan bahwa program LucaCadalora berhasil menentukan kendaraan yang diangkut beserta posisi dan orientasinya dengan tetap memenuhi batas geladak, kapasitas kapal, dan larangan tumpang tindih. Pada dataset yang diuji, Random Restart Hill Climbing dan Genetic Algorithm menunjukkan performa paling baik dan konsisten dengan mencapai nilai optimum 235 pada seluruh percobaan. Steepest Ascent, Stochastic, dan Sideways Move mencapai optimum pada dua dari tiga percobaan, sedangkan First Improvement hanya mencapainya pada satu percobaan. Simulated Annealing menghasilkan nilai yang konsisten sebesar 230 atau 97,87% dari nilai optimum. Hasil tersebut menunjukkan bahwa Random Restart mampu mengurangi ketergantungan terhadap state awal, sedangkan Genetic Algorithm dapat mengeksplorasi lebih banyak kandidat melalui pencarian berbasis populasi.

Dari segi durasi, varian Hill Climbing secara umum lebih cepat dibandingkan Simulated Annealing dan Genetic Algorithm. Random Restart membutuhkan waktu lebih lama daripada varian Hill Climbing lainnya karena menjalankan pencarian dari beberapa state awal, tetapi tetap memberikan hasil terbaik pada dataset ini. Penambahan jumlah generasi dan ukuran populasi pada Genetic Algorithm meningkatkan durasi pencarian, tetapi tidak meningkatkan nilai akhir karena seluruh konfigurasi telah mencapai optimum. Selain memenuhi algoritma wajib, program juga berhasil mengimplementasikan seluruh varian Hill Climbing, dua fungsi objektif alternatif, visualisasi hasil pencarian, serta penempatan kendaraan pada lebih dari satu kapal.

3.2 Saran

Berdasarkan hasil eksperimen tersebut, beberapa saran yang dapat diberikan adalah sebagai berikut.

1. Jika prioritas utama adalah memperoleh solusi berkualitas tinggi dalam waktu relatif singkat, Random Restart Hill Climbing dapat digunakan karena berhasil mencapai nilai optimum secara konsisten dengan durasi yang lebih rendah dibandingkan Genetic Algorithm.

2. Jika diperlukan eksplorasi ruang pencarian yang lebih luas, Genetic Algorithm dapat digunakan, terutama pada dataset yang lebih besar dan kompleks. Namun, ukuran populasi dan jumlah generasi perlu disesuaikan agar peningkatan durasi sebanding dengan kualitas solusi yang diperoleh.

3. Parameter temperatur awal, laju pendinginan, dan jumlah iterasi pada Simulated Annealing perlu diuji lebih lanjut karena konfigurasi yang digunakan belum berhasil mencapai nilai optimum.

4. Pengukuran durasi Genetic Algorithm sebaiknya dilakukan sejak pembentukan populasi awal agar waktu yang dicatat mencerminkan keseluruhan proses pencarian.

5. Pengujian selanjutnya sebaiknya menggunakan lebih banyak seed serta variasi ukuran kapal, jumlah kendaraan, kapasitas, dan distribusi shipping fee untuk menguji konsistensi dan skalabilitas setiap algoritma.

6. Fitur fungsi objektif alternatif dan multi-kapal dapat dikembangkan lebih lanjut menggunakan dataset yang lebih kompleks agar manfaatnya dalam skenario operasional dapat dianalisis secara lebih mendalam.

PEMBAGIAN TUGAS

Tabel 61. Pembagian Tugas Anggota Kelompok

Anggota	NIM	Pembagian Tugas
Muhammad Syauqi Azizi	18224046	Menyiapkan struktur awal proyek,mengintegrasikan algoritma melalui antarmuka baris perintah (CLI),mengimplementasikan visualisasi penempatan kendaraan dan grafik hasil pencarian, menyediakan ekspor hasil eksperimen,serta menambahkan pengujian modul visualisasi.
Rafi Azmi Zafarani	18224082	Mengimplementasikan model kapal, kendaraan, dan state, pemeriksaan batasan, fungsi objektif, dan pembangkitan tetangga,algoritma Hill Climbing beserta variasinya,pengujian modul inti dan Hill Climbing,serta pengembangan bonus optimasi multikapal.
Athar Ramadhana Dermawan	18224104	Mengimplementasikan algoritma Simulated Annealing dan Genetic Algorithm,menambahkan pengujian perilaku dan parameter kedua algoritma,serta menyediakan modul pelaksanaan untuk melakukan eksperimen,membuat sebagian besar laporan tugas besar 1 dasar AI.

LAMPIRAN PENGGUNAAN GENERATIVE AI

Area Gambar atau Lampiran

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:
Nama/NIM : Muhammad Syauqi Azizi (18224046); Rafi Azmi Zafarani (18224082); Athar Ramadhana Dermawan (18224104)
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika (STEI), ITB
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Inteligensi Artifisial
Judul Tugas/Proposal : Optimasi Penempatan Kendaraan pada Geladak Kapal Kargo dengan Algoritma Local Search

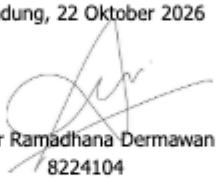
Menyatakan bahwa kami menggunakan AI sebagai alat bantu dalam pengerjaan dan penyusunan luaran tugas pada mata kuliah tersebut.

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	Tidak digunakan
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Terbatas: parafrase dan pengembangan teks
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Terbatas: diskusi teknis dan cross-check guideline
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	Tidak digunakan
Lainnya, Jelaskan:	Ya	Terbatas: pembuatan template, elaborasi, dan perapian dokumen

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Area Gambar atau Lampiran

Bandung, 22 Oktober 2026



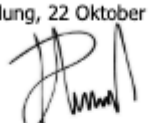
Athar Ramadhana Dermawan
8224104

Bandung, 22 Oktober 2026



Muhammad Syauqi Aziz
18224046

Bandung, 22 Oktober 2026



Rafi Azmi Zafarani
18224082